

T.C.
SAKARYA ÜNİVERSİTESİ
BİLGİSAYAR VE BİLİŞİM BİLİMLERİ FAKÜLTESİ

BSM 498 BİTİRME ÇALIŞMASI

EĞİTİM ALANINDA ÖZELLEŞTİRİLMİŞ RAG
TABANLI AKADEMİK CHATBOT

B221210037 – Egemen BOZCA

Fakülte Anabilim Dalı : BİLGİSAYAR MÜHENDİSLİĞİ
Tez Danışmanı : Prof. Dr. Cüneyt BAYILMIŞ

2025-2026 Bahar Dönemi

T.C.
SAKARYA ÜNİVERSİTESİ
BİLGİSAYAR VE BİLİŞİM BİLİMLERİ FAKÜLTESİ

EĞİTİM ALANINDA ÖZELLEŞTİRİLMİŞ RAG
TABANLI AKADEMİK CHATBOT

BSM 498 - BİTİRME ÇALIŞMASI

Egemen BOZCA

Fakülte Anabilim Dalı : BİLGİSAYAR MÜHENDİSLİĞİ

Bu tez .. / .. / ... tarihinde aşağıdaki jüri tarafından oybirliği / oyçokluğu ile kabul edilmiştir.

.....
Jüri Başkanı

.....
Üye

.....
Üye

ÖNSÖZ

Bu bitirme çalışması, eğitim kurumlarının bilgi yönetim süreçlerini yapay zeka teknolojileriyle modernize etme ihtiyacından doğmuştur. Günümüzde öğrencilerin ve akademik personelin bilgiye erişim taleplerinin artması, otomatize edilmiş ve bağlama duyarlı bir sistem gerekliliğini ortaya koymuştur.

Bu süreçte değerli rehberliğini ve akademik desteğini esirgemeyen danışmanım Prof. Dr. Cüneyt BAYILMIŞ'a sonsuz teşekkürlerimi sunarım. Ayrıca çalışmalarım boyunca her türlü desteği sağlayan aileme ve arkadaşlarıma minnettarım.

İÇİNDEKİLER

ÖNSÖZ	iii
İÇİNDEKİLER	iv
SİMGELER VE KISALTMALAR LİSTESİ.....	x
ŞEKİLLER LİSTESİ	xi
TABLolar LİSTESİ	xii
ÖZET	xiii
BÖLÜM 1. GİRİŞ.....	14
1.1. Problemin Tanımı	15
1.2. Çalışmanın Amacı ve Hedefleri	15
1.3. Başarı Ölçütleri.....	16
1.4. Çalışmanın Kapsamı ve Kısıtları.....	17
1.5. Çalışmanın Özgün Katkısı.....	18
1.6. Raporun Organizasyonu	18
BÖLÜM 2. İLGİLİ ÇALIŞMALAR	20
2.1. Büyük Dil Modelleri ve Akademik Chatbot Sistemleri	20
2.2. Retrieval-Augmented Generation (RAG).....	21
2.2.1. RAG Paradigmasının Temelleri.....	21
2.2.2. Belge Temelleme ve Halüsinasyon Önleme.....	21
2.2.3. Çoklu Belge RAG ve Dinamik Bağlam Yönetimi	22
2.3. Doğal Dilden SQL'e Dönüşüm (Metin-SQL).....	22
2.3.1. Genel Durum ve Kıyaslama Veri Kümeleri.....	22
2.3.2. İnce Ayarlı Modellerden LLM Tabanlı Yaklaşımlara	22
2.3.3. Anlamsal Örnek Seçimi ve Öz-Onarım.....	23
2.3.4. Türkçe Dilinin Metin-SQL Üzerindeki Etkileri	23
2.4. Niyet Sınıflandırma	24
2.4.1. Geleneksel Yaklaşımlardan BDM Tabanlı Sınıflandırmaya.....	24
2.4.2. Konuşma Bağlamının Sınıflandırmaya Etkisi	24
2.4.3. Çok Hatlı Sistemlerde Yönlendirme.....	25
2.5. Halüsinasyon Azaltma Yöntemleri	25
2.6. Mevcut Yaklaşımların Karşılaştırmalı Değerlendirmesi	26
BÖLÜM 3. SİSTEM MİMARİSİ	27

3.1.	Genel Mimari ve Katmanlı Yapı.....	27
3.2.	Yazılım Gereksinim Listesi	29
3.3.	Birimler Arası Bilgi Akışı.....	29
3.4.	Niyet Sınıflandırma ve Yönlendirme Motoru	30
3.4.1.	İki Kademeli Sınıflandırma Stratejisi.....	30
3.4.2.	Belge Yüklü Durum Önceliği.....	31
3.5.	Few-Shot Text-to-SQL Modülü.....	31
3.5.1.	Şema Enjeksiyonu	32
3.5.2.	Anlamsal Few-Shot Örnek Seçimi	32
3.5.3.	SQL Üretimi ve Normalleştirme	32
3.5.4.	Öz-Onarımlı Düzeltme Döngüsü.....	32
3.6.	RAG Belge Yöneticisi ve Halüsinasyon Önleme Politikası.....	33
3.6.1.	Belge İşleme Hattı	33
3.6.2.	Dinamik Chunk Bütçeleme.....	33
3.6.3.	Eşik Kapısı ve Sıfır-Halüsinasyon Politikası	34
3.7.	Web Araması Entegrasyonu	35
3.8.	Token ve Maliyet Takip Mekanizması	35
3.9.	Çoklu Oturum Yönetimi	35
3.10.	Kullanılan Teknolojiler ve Araçlar	36
BÖLÜM 4.	UYGULAMA.....	37
4.1.	Geliştirme Ortamı ve Kurulum.....	37
4.1.1.	Donanım ve Yazılım Altyapısı.....	37
4.1.2.	Bağımlılık Başlatma Stratejisi.....	38
4.2.	Veritabanı Kurulumu ve Şema Tasarımı.....	38
4.2.1.	Demo Veritabanının Oluşturulması	38
4.2.2.	Şema Enjeksiyon Mekanizması.....	39
4.2.3.	SQL Güvenliği.....	40
4.3.	Gömme Modeli ve Vektör Veritabanı	40
4.3.1.	Gömme Modeli Seçimi ve Gerekçesi	40
4.3.2.	ChromaDB Koleksiyon Yönetimi	40
4.4.	Flask API Mimarisi	41
4.4.1.	RESTful Endpoint Tasarımı	41
4.4.2.	Oturum Durumu Yönetimi	42

4.5.	Kullanıcı Arayüzü Tasarımı.....	42
4.5.1.	Üç Panelli Yerleşim Mimarisi	42
4.5.2.	Asenkron İletişim Altyapısı.....	45
4.5.3.	Markdown Render Desteği.....	45
4.6.	Web Sitesi Tarayıcı Bileşeni.....	45
4.6.1.	Tarayıcı Mimarisi	45
4.6.2.	Kısıtlamalar ve Hata Yönetimi	46
4.7.	Karşılaşılan Teknik Sorunlar ve Çözümleri.....	46
4.7.1.	ChromaDB Kalıcı Depolama Uyumsuzlukları.....	46
4.7.2.	ILIKE / Tırnak İşareti Uyumsuzlukları	46
4.7.3.	Gömme Modeli İlk Yükleme Gecikmesi.....	46
4.7.4.	Çoklu Sohbet Oturumlarında Durum Karışması	47
4.8.	Güvenlik Değerlendirmesi	47
4.8.1.	API Anahtar Güvenliği	47
4.8.2.	Dosya Yükleme Güvenliği	47
4.8.3.	SQL Enjeksiyon Önlemi.....	47
BÖLÜM 5.	DENEYSEL DEĞERLENDİRME.....	49
5.1.	Test Ortamı ve Metodoloji.....	49
5.1.1.	Donanım ve Yazılım Konfigürasyonu	49
5.1.2.	Değerlendirme Metrikleri.....	49
5.2.	Text-to-SQL Başarım Değerlendirmesi.....	50
5.2.1.	Test Veri Kümesinin Tasarımı	50
5.2.2.	Karşılaştırmalı Sonuçlar	50
5.2.3.	Bulguların Yorumu	51
5.3.	RAG Belge Temelleme Değerlendirmesi	51
5.3.1.	Test Tasarımı.....	51
5.3.2.	Sonuçlar.....	52
5.3.3.	Temel Sistem ile Karşılaştırma.....	52
5.3.4.	Eşik Değeri Analizi	53
5.4.	Niyet Sınıflandırma Değerlendirmesi	53
5.4.1.	Test Veri Kümesi	53
5.4.2.	Sonuçlar.....	54
5.4.3.	Konuşma Geçmişinin Etkisi.....	54

5.4.4.	Anahtar Kelime Ön Filtresinin Katkısı	54
5.5.	Maliyet Analizi	54
5.5.1.	Metodoloji	54
5.5.2.	Model Başına Maliyet Karşılaştırması	55
5.5.3.	Sorgu Türü Bazında Token Dağılımı.....	55
5.5.4.	Oturum Başlığı Üretiminin Maliyete Etkisi	55
5.6.	Yanıt Süresi Analizi	56
BÖLÜM 6.	TARTIŞMA	57
6.1.	Deneyisel Bulguların Yorumu	57
6.1.1.	Text-to-SQL Başarımının Değerlendirilmesi	57
6.1.2.	Halüsinasyon Önleme Politikasının Etkinliği	58
6.1.3.	Niyet Sınıflandırmanın Sınır Durumları.....	58
6.1.4.	Maliyet ve Gecikme Dengesi	59
6.2.	Tasarım Kararlarının Değerlendirilmesi.....	59
6.2.1.	Flask Seçiminin Gerekçesi	59
6.2.2.	SQLite Seçiminin Gerekçesi	60
6.2.3.	Çevrimdışı Gömme Modeli Kararı.....	60
6.3.	Sistemin Güçlü Yönleri	60
6.4.	Sistemin Kısıtları	61
6.4.1.	Oturum Kalıcılığı Sorunu	61
6.4.2.	Tek İş Parçacıklı Sunucu Kısıtı	62
6.4.3.	Küçük Değerlendirme Kümesi	62
6.4.4.	Az Sayıda Few-Shot Örneği.....	62
6.4.5.	DuckDuckGo Bağımlılığı.....	62
6.5.	Tasarım-Uygulama Uyumsuzlukları	63
6.6.	Literatürle Karşılaştırmalı Değerlendirme	63
BÖLÜM 7.	SONUÇLAR VE ÖNERİLER	66
7.1.	Temel Bulgular	66
7.2.	Özgün Katkıların Değerlendirilmesi	67
7.3.	Araştırma Sorularına Yanıtlar	68
7.4.	Gelecek Çalışma Önerileri	69
7.5.	Çalışmanın Genel Değerlendirmesi.....	70
7.6.	Sonuç	71

BÖLÜM 8.	DEĞİŞİKLİK YÖNETİMİ VE MÜHENDİSLİK STANDARTLARI	72
8.1.	Değişiklik Yönetimi Süreci	72
8.1.1.	Değişiklik Talep Sınıflandırması	72
8.1.2.	Gerçekleştirilen Başlıca Değişiklikler	73
8.2.	Sürüm Kontrolü ve Kod Yönetimi	73
8.2.1.	Git Dal Stratejisi	73
8.2.2.	Yedekleme ve Süreklilik	74
8.3.	Mühendislik Standartları ve Kod Kalitesi	74
8.3.1.	Python Kod Yazım Standardı	74
8.3.2.	Hata Yönetimi Politikası	75
8.3.3.	Güvenlik Standartları	75
8.3.4.	Bağımlılık Yönetimi	76
8.4.	Yazılım Mimarisi Standartları	76
8.4.1.	SOLID Prensipleri	76
8.4.2.	Uygulanan Tasarım Desenleri	77
8.5.	Test ve Doğrulama Süreçleri	77
8.5.1.	Test Katmanları	77
8.5.2.	Test Veri Kümeleri	78
8.5.3.	Regresyon Kontrolü	78
BÖLÜM 9.	TİCARİLEŞME PLANI	79
9.1.	Pazar Analizi ve Hedef Kitle	79
9.1.1.	Pazar Büyüklüğü ve Fırsat	79
9.1.2.	Hedef Pazar Kademeleri	79
9.1.3.	Rekabet Analizi	80
9.2.	Ürün Olgunlaştırma Yol Haritası	80
9.3.	İş Modeli ve Gelir Stratejisi	80
9.3.1.	Gelir Akışları	80
9.3.2.	Beş Yıllık Gelir Projeksiyonu	81
9.4.	Finansman ve Büyüme Stratejisi	81
9.4.1.	Başlangıç Finansmanı	81
9.4.2.	Büyüme Aşamaları	82
9.5.	Fikri Mülkiyet Stratejisi	82
9.6.	Risk Analizi ve Azaltma Stratejileri	83

ÖZGEÇMİŞ	87
BSM498 BİTİRME ÇALIŞMASI.....	88
DEĞERLENDİRME VE SÖZLÜ SINAV TUTANAĞI.....	88

SİMGELER VE KISALTMALAR LİSTESİ

API	Application Programming Interface (Uygulama Programlama Arayüzü)
BDM	Büyük Dil Modeli
ChromaDB	Açık kaynaklı vektör veritabanı sistemi
DFD	Data Flow Diagram (Veri Akış Diyagramı)
GPT	Generative Pre-trained Transformer
JSON	JavaScript Object Notation
LLM	Large Language Model (Büyük Dil Modeli)
NLP	Natural Language Processing (Doğal Dil İşleme)
OBS	Öğrenci Bilgi Sistemi
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SQL	Structured Query Language (Yapısal Sorgulama Dili)
SDLC	Software Development Life Cycle
SOLID	Single Responsibility, Open/Closed, Liskov, Interface, Dependency
UI	User Interface (Kullanıcı Arayüzü)
USD	United States Dollar (Amerikan Doları)

ŞEKİLLER LİSTESİ

Şekil 3.1. Sistem Mimarisi Blok Şeması	28
Şekil 3.2 Niyet Yönlendirme Akış Şeması	31
Şekil 3.3 Few-Shot SQL Pipeline	33
Şekil 3.4 RAG Pipeline.....	34
Şekil 4.1 ER Diyagramı	39
Şekil 4.2. Web Scrape Sorgusu.....	43
Şekil 4.3. RAG Modunda Çoklu Belge Sorgusu	43
Şekil 4.4. Text-to-SQL Sorgusu ve Veri Tabanı Yanıtı.....	44
Şekil 4.5. Kapsam Dışı Sorguya Ret Yanıtı.....	44
Şekil 4.6. Web Araması ve Çok Sağlayıcı Maliyet Karşılaştırması	44

TABLÖLAR LİSTESİ

Tablo 4.1. Demo Veri Tabanı Şeması ve Kayıt Sayıları	39
Tablo 5.1. Text-to-SQL Yürütme Doğruluğu Karmaşıklık Bazında.....	51
Tablo 5.3. RAG Belge Modu Değerlendirme Sonuçları	52
Tablo 5.4. RAG vs. Temel Model Karşılaştırması	53
Tablo 5.6. Model Başına Ortalama Maliyet (100 etkileşim).....	55
Tablo 5.7. Bileşen Bazında Ortalama Yanıt Süreleri	56
Tablo 6.1. Sistemin Temel Kısıtları ve Önerilen Çözümler	65
Tablo 8.1. Geliştirme Sürecindeki Temel Değişiklik Kararları	73
Tablo 9.1. Ürün Olgunlaştırma Aşamaları	80
Tablo 9.2. Beş Yıllık Gelir Projeksiyonu (Yalnızca SaaS Geliri).....	81
Tablo 9.3. Risk Matrisi ve Azaltma Stratejileri.....	83

ÖZET

Anahtar Kelimeler: Retrieval-Augmented Generation (RAG), Text-to-SQL, Büyük Dil Modeli (LLM), Niyet Sınıflandırma, Ajansal Chatbot

Bu proje, eğitim kurumları için geliştirilmiş ajansal bir RAG tabanlı chatbot sistemidir. Sistemin temel yeniliği, her kullanıcı sorgusunu beş kategoriye (DB_QUERY, SEARCH, GENERAL, META, DOCUMENT) sınıflandıran niyet tabanlı yönlendirme mimarisinde yatmaktadır. Sistem bu sınıflandırmaya göre sorguyu veri tabanı, belge, internet araması veya genel sohbet pipeline'ına yönlendirmektedir.

Veri tabanı sorguları için ChromaDB ve çok dilli bir embedding modeli (multilingual-e5-small) kullanılarak anlamsal olarak en benzer soru-SQL çiftleri dinamik olarak seçilmekte ve few-shot prompt oluşturulmaktadır. SQL üretiminde hata oluşursa sistem otomatik olarak self-healing mekanizmasını devreye sokmaktadır. Belge modunda ise yüklenen PDF ve Excel dosyalarından bilgi çekilmekte; benzerlik eşiği ($\tau=0.45$) altında kalan sorgulara açıkça kapsam dışı yanıt verilmektedir.

DeneySEL sonuçlar sistemin etkinliğini doğrulamaktadır: few-shot Text-to-SQL yaklaşımı %90 SQL doğruluğu elde ederken, zero-shot baseline yalnızca %67 başarı göstermektedir. Belge modu %100 kapsam dışı ret oranına ulaşmış, niyet sınıflandırması ise %92 doğrulukla çalışmaktadır. Sistem GPT-4o-mini ve Gemini Flash modellerini desteklemekte, Flask tabanlı bir web arayüzü üzerinden çoklu oturum yönetimi sunmaktadır.

BÖLÜM 1. GİRİŞ

Bilgi teknolojilerinin akademik kurumlarla kesiştiği noktada, üniversite bilgi sistemleri son yıllarda ciddi bir dönüşüm baskısıyla karşı karşıya kalmıştır. Öğrenci kayıtları, not sistemleri, ders programları, akademisyen iletişim bilgileri ve kurumsal belgeler gibi heterojen veri kaynaklarına erişim, günümüzde hâlâ teknik bilgi gerektiren, çok adımlı ve zaman alıcı süreçlere dayanmaktadır. Bir öğrencinin dönem not ortalamasını öğrenmek ya da belirli bir dersin sorumlu akademisyenini bulmak için bile sistemler arası geçiş yapmak gerekmektedir. Bu durum hem kullanıcı deneyimini zayıflatmakta hem de kurumsal bilgiye erişimdeki eşitsizlikleri derinleştirmektedir.

Büyük Dil Modelleri'nin (BDM) ortaya çıkışı, bu problemi yeniden çerçevelemek için güçlü bir araç sunmaktadır. GPT-4o, Gemini Flash gibi günümüz dil modelleri; doğal dilde verilen sorulara tutarlı ve bağlama duyarlı yanıtlar üretebilmektedir. Ancak bu modellerin kurumsal ortamlarda doğrudan kullanılması ciddi kısıtlar doğurmaktadır: parametrik bilginin güncelliğini yitirmesi, halüsinasyon eğilimi, canlı veri tabanlarını sorgulayamama ve kapalı alan belgelerinden bilgi çekememe bunların başında gelmektedir.

Bu kısıtları gidermek amacıyla geliştirilen Retrieval-Augmented Generation (RAG) paradigması, üretim sürecine harici bilgi kaynaklarını dinamik olarak entegre ederek modelin olgusal tabanını güçlendirmektedir. Buna paralel olarak Metin-SQL teknolojisi, doğal dil sorgularını doğrudan çalıştırılabilir SQL komutlarına dönüştürerek veri tabanı erişimini demokratikleştirmektedir. Ancak bu iki paradigmanın ayrı ayrı uygulanması, gerçek akademik kullanım senaryolarının karmaşıklığını karşılamakta yetersiz kalmaktadır. Kullanıcıların aynı sohbet akışı içinde hem yapılandırılmış veri tabanı sorguları hem de yüklenen belgeler üzerinde soru sorabilmesi hem web araması yapabilmesi hem de genel sohbet yürütebilmesi için bütünlük bir mimari gerekmektedir.

Bu çalışma, söz konusu ihtiyacı karşılamak üzere tasarlanmış ajansal, niyet-yönlendirmeli ve çok modlu bir akademik chatbot sistemi sunmaktadır.

1.1. Problemin Tanımı

Sakarya Üniversitesi Bilgisayar Mühendisliği bölümü özelinde ele alındığında, akademik bilgiye erişimdeki temel sorun üç ayrı boyutta kendini göstermektedir.

Birinci boyut teknik bariyerlerdir. Öğrenci Bilgi Sistemi (OBS) gibi mevcut sistemler SQL bilgisi gerektiren sorgu arabirimleri sunmakta ya da yalnızca önceden tanımlanmış filtreler üzerinden çalışmaktadır. "Bu dönem kaç öğrenci bütünlemeye kaldı?" gibi basit bir soruyu yanıtlamak için bile tablolar arası JOIN işlemleri bilmek gerekmektedir. Bu durum teknik olmayan kullanıcıları sistematik biçimde dışlamaktadır.

İkinci boyut bilgi dağınıklığıdır. Akademik mevzuat, sınav tarihleri, burs duyuruları ve ders içerikleri çoğunlukla farklı belge formatlarında (PDF, Excel, Word) ve farklı platformlarda dağınık hâlde bulunmaktadır. Kullanıcı, bir sorunun cevabını bulmak için birden fazla kaynağa bakmak zorunda kalmaktadır.

Üçüncü boyut güncellik sorunudur. Parametrik LLM'ler eğitim verilerinin kesim tarihinden sonraki bilgilere erişememektedir. Akademik takvim, dönemlik ders açılışları veya değişen not sistemleri gibi dinamik bilgiler bu modeller tarafından doğru yanıtlanamaz.

Mevcut çözümler bu üç boyutu bir arada ele alamamaktadır. Genel amaçlı chatbot platformları (ChatGPT, Copilot) kurumsal veritabanlarına bağlanamaz. Klasik OBS sistemleri doğal dil arayüzü sunmaz. Bilgi tabanı araçları ise yalnızca belge sorgulama yapabilir, SQL üretemez. Bu çalışma tam da bu boşluğu doldurmayı hedeflemektedir.

1.2. Çalışmanın Amacı ve Hedefleri

Bu çalışmanın birincil amacı, akademik bilgi erişim süreçlerini tek bir birleşik doğal dil arayüzü altında toplayan, niyet-yönlendirmeli ve halüsinasyona dayanıklı bir chatbot sistemi geliştirmektir.

Bu temel amaca ulaşmak için belirlenen alt hedefler şunlardır:

Birinci hedef, doğal dil ile veri tabanı sorgulama yeteneği kazandırmaktır. Kullanıcı "Bu dönem kaç öğrenci dersten kaldı?" gibi Türkçe bir soru sorduğunda sistem, arka planda doğru SQL sorgusunu üretmeli, çalıştırmalı ve sonucu anlaşılır bir dille sunmalıdır.

İkinci hedef, belgeye dayalı soru-cevap altyapısı oluşturmaktır. Kullanıcının yüklediği PDF veya Excel belgelerinden ilgili bölümler çekilerek yanıt üretilmeli; belge bu konuyu kapsamamıyorsa sistem açıkça bunu belirtmelidir.

Üçüncü hedef, niyet tabanlı otomatik yönlendirme mekanizması kurmaktır. Her kullanıcı ifadesi beş kategoriden birine (DB_QUERY, DOCUMENT, SEARCH, GENERAL, META) atanmalı ve uygun işlem hattına yönlendirilmelidir.

Dördüncü hedef, gerçek zamanlı web araması entegrasyonu sağlamaktır. Güncel bilgi gerektiren sorular için internet arama motoru sonuçları özetlenerek yanıtta dahil edilmelidir.

Beşinci hedef, çoklu BDM desteği ve maliyet şeffaflığıdır. GPT-4o-mini ve Gemini Flash modellerinin karşılaştırmalı kullanımı ile her sorgu için harcanan token miktarı ve tahmini USD maliyeti kullanıcıya anlık olarak gösterilmelidir.

1.3. Başarı Ölçütleri

Geliştirilen sistemin başarısı dört temel metrik üzerinden değerlendirilmiştir.

Sorgu doğruluğu açısından hedef, few-shot Text-to-SQL yaklaşımının sıfır-atışlı temele kıyasla ölçülebilir bir iyileşme sağlamasıdır. Gerçekleştirilen değerlendirmede

sıfır-atışlı yöntemin %67 başarımlı gösterdiği durumda, few-shot RAG yaklaşımının %90'a ulaşması hedeflenmiş ve bu hedefe ulaşılmıştır.

Halüsinasyon önleme açısından hedef, belge modunda kapsam dışı sorguların %100 oranında reddedilmesidir. Sistem bu hedefe tam olarak ulaşmıştır; hiçbir kapsam dışı sorguda uydurma yanıt üretilmemiştir.

Niyet sınıflandırma başarımlı açısından hedef, 50 test ifadesinden oluşan değerlendirme kümesinde %90 ve üzeri doğruluk elde etmektir. Sistem %92 doğrulukla bu hedefi aşmıştır.

Sistem yanıt süresi açısından hedef, kullanıcı deneyimini olumsuz etkilemeyecek kabul edilebilir gecikme süreleridir. Değerlendirme ortamında sorgu türüne bağlı olarak ortalama yanıt süresi 1,1 ile 3,6 saniye arasında ölçülmüş; tüm bileşenlerde belirlenen 5 saniyelik üst sınırın altında kalmıştır.

1.4. Çalışmanın Kapsamı ve Kısıtları

Bu çalışma; beş modüllü bir sistem mimarisi geliştirmeyi, few-shot RAG tabanlı Text-to-SQL üretimini, benzerlik eşiği kontrollü belge RAG'ını, DuckDuckGo web araması entegrasyonunu ve Flask tabanlı üç panelli bir web arayüzünü kapsamaktadır.

Çalışmanın kapsam sınırları şu şekilde belirlenmiştir: Veri tabanı olarak Sakarya Üniversitesi Bilgisayar Mühendisliği bölümü yapısını yansıtan 6 tablolü sentetik bir SQLite veri tabanı kullanılmıştır. Few-shot örnek kümesi 7 el yapımı soru-SQL çiftiyle sınırlı tutulmuştur. Değerlendirme veri kümeleri 30 (SQL), 20 (RAG) ve 50 (niyet) test örneğinden oluşmaktadır. Web araması bileşeni ticari bir API yerine ücretsiz DuckDuckGo servisi üzerine inşa edilmiş olup hız sınırlaması içermektedir.

Bu kısıtlar, mimarinin teknik doğruluğunu kanıtlamak açısından yeterli olup gerçek kurumsal veri tabanları üzerinde daha kapsamlı değerlendirmeler gelecek çalışmalar için önerilmektedir.

1.5. Çalışmanın Özgün Katkısı

Mevcut akademik chatbot çalışmaları incelendiğinde, sistemlerin büyük çoğunluğunun ya yalnızca RAG ya da yalnızca Text-to-SQL üzerine odaklandığı görülmektedir. Her iki paradigmayı, web araması ve genel sohbet yeteneğiyle birleştiren bütünleşik ve niyet-yönlendirmeli mimariler literatürde oldukça sınırlı kalmaktadır.

Bu çalışmanın dört özgün katkısı bulunmaktadır. Birincisi, sıfır-halüsinasyon temelleme politikasıdır: sistem belge modunda çalışırken parametrik bilgisini hiçbir koşulda yanıtı katmaz; benzerlik eşiği kapısı bu politikayı deterministik biçimde uygular. İkincisi, dinamik k ile çoklu belge RAG'ıdır: birden fazla belge yüklendiğinde bağlam penceresi her belgeye eşit pay verilerek yönetilir. Üçüncüsü, öz-onarımlı Metin-SQL hattıdır: SQL yürütme hatası durumunda hata mesajı ve özgün SQL tekrar modele beslenip tek seferlik onarım denenir; bu mekanizma hataların %80'ini gidermektedir. Dördüncüsü, gerçek zamanlı çok-sağlayıcı maliyet karşılaştırmasıdır: her yanıt için token tüketimi ve tahmini USD maliyet, farklı sağlayıcılar arasında anlık olarak karşılaştırılabilir hâlde sunulur.

1.6. Raporun Organizasyonu

Bu rapor dokuz ana bölüm halinde düzenlenmiştir. İkinci bölümde RAG sistemleri, Text-to-SQL yaklaşımları, niyet sınıflandırma ve halüsinasyon azaltma konularındaki ilgili çalışmalar incelenmektedir. Üçüncü bölümde sistem mimarisi ve her bileşenin tasarım kararları ayrıntılandırılmaktadır. Dördüncü bölümde gerçekleştirme detayları, teknoloji yığını ve demo veri tabanı açıklanmaktadır. Beşinci bölümde deneysel değerlendirme sonuçları sunulmaktadır. Altıncı bölümde sistemin güçlü ve zayıf yönleri, tasarım kararlarının gerekçeleri ve literatürle karşılaştırmalı değerlendirme ele alınmaktadır. Yedinci bölümde temel bulgular özetlenmekte, özgün katkılar vurgulanmakta ve gelecek çalışma önerileri sunulmaktadır. Sekizinci bölümde geliştirme süreci boyunca uygulanan değişiklik yönetimi politikaları ve mühendislik

standartları aktarılmaktadır. Dokuzuncu bölümde ise sistemin ticarileşme planı, pazar analizi ve risk değerlendirmesi yer almaktadır.

BÖLÜM 2. İLGİLİ ÇALIŞMALAR

Yapay zekâ destekli bilgi erişim sistemleri, son yıllarda Doğal Dil İşleme (DNİ), büyük dil modelleri ve vektör tabanlı arama teknolojilerindeki hızlı gelişmelerle birlikte köklü bir dönüşüm geçirmektedir. Bu bölümde, geliştirilen sistemin dayandığı dört temel araştırma alanı Retrieval-Augmented Generation (RAG), Metin-SQL ve few-shot yönlendirmesi, niyet sınıflandırma ve BDM'lerde halüsinasyon azaltma kapsamlı biçimde incelenmekte; mevcut çalışmalar değerlendirilerek önerilen sistemin bu literatürdeki konumu ortaya konulmaktadır.

2.1. Büyük Dil Modelleri ve Akademik Chatbot Sistemleri

Büyük Dil Modelleri (BDM), milyarlarca parametrelili dönüştürücü (transformer) mimarileri üzerine inşa edilerek devasa metin kümeleriyle önceden eğitilmiş yapay zekâ sistemleridir. GPT-4o ve Gemini gibi günümüz modelleri; doğal dil anlama, bağlamsal akıl yürütme ve kod üretimi konularında insan düzeyine yakın başarımlar sergilemektedir. Ancak bu modellerin kapalı alan kurumsal ortamlarda kullanılması, iyi belgelenmiş bir dizi kısıtlamayı da beraberinde getirmektedir: eğitim kesim tarihinden sonraki bilgilere erişilememesi, halüsinasyon eğilimi ve canlı ilişkisel veritabanlarını doğrudan sorgulayamama bunların en kritikleridir.

Akademik ortamlarda BDM tabanlı chatbot geliştirme çalışmaları giderek yaygınlaşmaktadır. Pasquarelli ve ark. (2024), üniversite öğrencilerine yönelik RAG tabanlı bir destek sistemi geliştirerek %82 kullanıcı memnuniyeti bildirmiştir [8]. Burgan ve ark. (2024), LangChain çerçevesini kullanan uyarlamalı bir BDM chatbotu geliştirerek üniversiteye özgü sorularda rekabetçi doğruluk elde etmiştir [19]. Dayarathne ve ark. (2025) ise bilgisayar bilimi literatürüne yönelik RAG tabanlı soru yanıtlamada birden fazla BDM'yi karşılaştırarak gömme modeli seçiminin kritik bir başarımlar faktörü olduğunu belirlemiştir [18]. Mevcut çalışma bu araştırmalardan, yalnızca belge sorgulama ile sınırlı kalmayıp Metin-SQL üretimi,

web araması ve niyet yönlendirmesini tek bir ajansal çerçeve altında bütünleştirmesiyle ayrılmaktadır.

2.2. Retrieval-Augmented Generation (RAG)

2.2.1. RAG Paradigmasının Temelleri

P. Lewis, E. Perez ve A. Piktus (2020) tarafından önerilen RAG mimarisi, bilgi yoğun NLP görevlerinde parametrik olmayan hafızanın parametrik BDM'lerle entegrasyonunu ilk kez sistematik biçimde ele almıştır [13]. Temel fikir; sorgu zamanında ilgili belge parçacıklarını (chunk) yoğun bir vektör gösterimi aracılığıyla bir bilgi tabanından geri çekerek bunları BDM bağlamına dinamik olarak enjekte etmektir. Bu sayede model, eğitim verilerinde yer almayan güncel ve alana özgü bilgilere dayalı yanıtlar üretebilmektedir.

Gao ve ark. (2023), kapsamlı bir inceleme çalışmasında RAG mimarilerini üç nesle ayırmıştır: Naive RAG, Advanced RAG ve Modular RAG [15]. Naive RAG, doğrudan geri alma ve üretim zincirinden oluşurken; Advanced RAG, sorgu yeniden yazma, hiyerarşik indeksleme ve çapraz kodlayıcı yeniden sıralama gibi tekniklerle bu temel yaklaşımın sınırlarını aşmaktadır. Modular RAG ise farklı görevler için bağımsız olarak değiştirilebilen bileşenlerden oluşan esnek bir yapı sunmaktadır. Bu çalışmada benimsenen yaklaşım, benzerlik eşiği kontrolü ve dinamik chunk bütçelemesi özellikleriyle Advanced RAG kategorisine girmektedir.

2.2.2. Belge Temelleme ve Halüsinasyon Önleme

Saf BDM çıkarımına kıyasla RAG'ın olgusal halüsinasyonları kayda değer ölçüde azalttığı gösterilmiştir [4]. Ancak bu iyileştirme koşulsuz değildir: alınan pasajlar kullanıcı sorgusunu karşılamadığında model, geri alma sonuçlarını görmezden gelerek parametrik bilgisine başvurabilmektedir. Bu durum literatürde "dışsal halüsinasyon" olarak adlandırılmaktadır [16].

Mevcut çalışmalar bu sorunu çeşitli biçimlerde ele almıştır. Self-RAG (Asai ve ark., 2024), modeli kendi geri alma kararlarını değerlendirmeye yönlendirirken [21]; ROWEN (Ding ve ark., 2025), uyarlamalı geri alma mekanizmaları önermiştir [16]. HalluGuard (Li ve ark., 2025) ise RAG çıktılarındaki halüsinasyon edilmiş iddiaları tespit etmek için küçük akıl yürütme modelleri kullanmaktadır [20]. Önerilen sistem bu yaklaşımlardan farklı olarak yapısal temellemeyi tercih etmektedir: belge modunda BDM'nin parametrik bilgisini yanıtı katması $\tau=0.45$ benzerlik eşiği kapısıyla deterministik biçimde engellenmektedir. Bu, model öz-denetimine değil donanımsal kısıtlamaya dayanan daha öngörülebilir bir halüsinasyon önleme stratejisidir.

2.2.3. Çoklu Belge RAG ve Dinamik Bağlam Yönetimi

Rafidhul ve ark. (2025), yapılandırılmamış belgeler için RAG'ı ChromaDB vektör deposuyla birleştiren bütünsel bir chatbot önererek bu çalışmayla doğrudan örtüşen bir mimari sergilemiştir [6]. Bununla birlikte, birden fazla belge yüklendiğinde bağlam penceresini verimli yönetmek ayrı bir problem oluşturmaktadır. Çoğu çalışma sabit k değeriyle geri alma yaparken, önerilen sistem belge sayısına göre her belgeye dinamik olarak $k = \lfloor K_{\max} / n_{\text{belge}} \rfloor$ chunk bütçesi ayırarak bağlam taşmasını önlemektedir. Bu, literatürde yaygın biçimde ele alınmamış ama pratik dağıtımlar için kritik bir tasarım kararıdır.

2.3. Doğal Dilden SQL'e Dönüşüm (Metin-SQL)

2.3.1. Genel Durum ve Kıyaslama Veri Kümeleri

Doğal dili SQL'e çevirmek (NL2SQL), veritabanı araştırmalarının köklü ve hâlâ aktif bir problemidir. Yu ve ark. (2018) tarafından yayımlanan Spider veri kümesi, karmaşık ve çok alanlı Metin-SQL değerlendirmesinde standart kıyaslama ölçütü hâline gelmiştir [1]. Bu veri kümesi; 10.000'den fazla soru, 200'den fazla veritabanı şeması ve birden fazla tabloyu birleştiren karmaşık sorgular içermesiyle alandaki en zorlu benchmark olma özelliğini korumaktadır.

2.3.2. İnce Ayarlı Modellerden LLM Tabanlı Yaklaşımlara

PICARD (Scholak ve ark., 2021), kısıtlı otomatik regresif kod çözümle güçlü sonuçlar elde etmiş önemli bir ince ayarlı sistemdir [5]. RESDSQL ise şema bağlamayı ön işleme adımına taşıyarak farklılaşmıştır. Bu ince ayarlı yaklaşımlar, eğitim verilerinde görülen şemalarda yüksek başarımlar gösterse de yeni alanlara aktarım gücüyle yaşamaktadır.

LLM'lerin ortaya çıkmasıyla birlikte few-shot bağlam içi öğrenme, birçok ince ayarlı modeli geride bırakmıştır. DIN-SQL (Pourreza ve Rafiei, 2023), öz-düzeltilmeli parçalı bağlam içi öğrenme yaklaşımıyla Spider üzerinde %82.8 yürütme doğruluğu elde etmiştir [7]. Sun ve ark. (2023), dikkatli istem tasarımının tek başına Spider'da %84.4 yürütme doğruluğuna ulaşabileceğini göstermiştir [3]. Bu çalışmadaki few-shot RAG yaklaşımı, genel amaçlı kıyaslama veri kümelerini değil Sakarya Üniversitesi'ne özgü küçük ve alana uyarlanmış bir şemayı hedeflemekte; bu nedenle daha az örnekle daha yüksek bağlama özgü doğruluk elde etmektedir.

2.3.3. Anlamsal Örnek Seçimi ve Öz-Onarım

Few-shot başarımının kilit belirleyicisi, hangi örneklerin sunulduğudur. Liu ve ark. (2022), en yakın komşu eğitim örneklerini geri alarak bağlam içi uygunluğu artıran kNN destekli örnek seçimini (KATE) önermiştir [12]. Bu ilkeyi önerilen sistem, LangChain'in SemanticSimilarityExampleSelector bileşeni aracılığıyla intfloat/multilingual-e5-small çok dilli gömme modeli ve ChromaDB üzerinde uygulamaktadır.

Öz-onarım mekanizması açısından SQLPrompt (Sun ve ark., 2023), yürütme hatalarını yeniden istem döngülerine entegre etmiştir [3]. Önerilen sistemde bu mekanizma tek adıma indirilmiş; hatalı SQL ve hata mesajı doğrudan modele beslenerek sonsuz döngü riski ortadan kaldırılmaktadır. Değerlendirme sonuçları bu tek adımlı onarımın başlangıç yürütme hatalarının %80'ini giderdiğini göstermektedir.

2.3.4. Türkçe Dilinin Metin-SQL Üzerindeki Etkileri

Metin-SQL literatürünün büyük bölümü İngilizce sorgular üzerine yoğunlaşmaktadır. Türkçe ise sondan eklemeli (agglutinative) morfolojik yapısıyla İngilizceden köklü biçimde ayrılmaktadır: aynı kök sözcük onlarca farklı ek alarak anlam değiştirebilmekte, bu durum sözcüklerin veritabanı varlıklarıyla eşleştirilmesini güçleştirmektedir. Örneğin "öğrencilerin notlarını listele" ifadesindeki "-lerin" tamlayan eki ve "-ini" belirtme ekinin ayrıştırılarak doğru JOIN koşuluna dönüştürülmesi gereklidir. Bu çalışmada çok dilli gömme modeli ve Türkçe sistem yönergeleri bu zorlukla başa çıkmak için kullanılmıştır; Türkçe Metin-SQL değerlendirmesinin kapsamlı biçimde ele alındığı çalışmalar ise gelecek araştırmalar için önemli bir boşluk oluşturmaktadır.

2.4. Niyet Sınıflandırma

2.4.1. Geleneksel Yaklaşımlardan BDM Tabanlı Sınıflandırmaya

Konuşma tabanlı sistemlerde kullanıcı niyetinin doğru tespiti, yanıt kalitesinin temel belirleyicisidir. Geleneksel yaklaşımlar BERT tabanlı ince ayarlı sınıflandırıcılara dayanmaktaydı [17]; bu modeller yüksek doğruluk sunsa da etiketlenmiş veri gerektiriyor ve yeni niyet kategorilerine uyum sağlamakta güçlük çekiyordu.

Son çalışmalar BDM'lerin etiketli veri olmaksızın rekabetçi sıfır-atışlı veya few-shot niyet sınıflandırması yapabildiğini göstermektedir [19]. Weld ve ark. (2022), ortak niyet tespiti ve slot doldurma modellerini kapsayan geniş bir inceleme sunarak cümle dönüştürücü tabanlı yaklaşımların geleneksel yöntemleri geride bıraktığını ortaya koymuştur [2]. López ve Chalendar (2025), çok taraflı konuşmalarda BDM'leri kullanarak kapsam dışı tespit ile niyet tanımayı birleştiren bir yaklaşım geliştirmiştir [22].

2.4.2. Konuşma Bağlamının Sınıflandırmaya Etkisi

Takip sorgularında açık varlık referansları çoğunlukla bulunmamaktadır. "Peki ya o öğrenci?" gibi bir ifadenin doğru sınıflandırılabilmesi için önceki konuşma turlarının

bağlam olarak kullanılması gerekmektedir. Önerilen sistem, son beş mesajı sınıflandırma istemine dahil ederek takip sorgularındaki doğruluğu %67'den %70'e yükseltmektedir. Bu bulgu, çok turlu konuşma bağlamının niyet sınıflandırma başarımı üzerindeki kritik etkisini doğrulamaktadır.

2.4.3. Çok Hatlı Sistemlerde Yönlendirme

Ma (2025), kullanıcılar ile arka uç hizmetler arasında niyet tabanlı sorgu yönlendirme yaklaşımını önermiş ve bu desen önerilen sistemin mimarisinde doğrudan benimsenmiştir [11]. Önerilen sistemdeki iki kademeli strateji anahtar kelime ön filtresi ardından LLM sınıflandırma gereksiz API çağrılarını azaltarak hem gecikme süresini hem de maliyeti optimize etmektedir. Bu hibrit yaklaşım, saf LLM sınıflandırmasına kıyasla %15 daha az API çağrısıyla aynı doğruluk düzeyine ulaşmaktadır.

2.5. Halüsinasyon Azaltma Yöntemleri

BDM'lerde halüsinasyon; modellerin olgusal desteği bulunmayan, ancak makul görünen metinler üretmesi durumunda ortaya çıkmakta. Ji ve ark. (2023) bu olguyu kapsamlı biçimde incelemiş; halüsinasyonu "içsel" (modelin eğitim verilerindeki tutarsızlıklardan kaynaklanan) ve "dışsal" (modelin aldığı bağlamdan uzaklaşmasıyla ortaya çıkan) olmak üzere iki kategoride ele almıştır [14].

RAG içsel halüsinasyonu geri alınan kanıtlara dayalı yanıtlar üreterek azaltmaktadır; ancak dışsal halüsinasyon modelin geri alınan bağlamı görmezden gelmesi hâlâ önemli bir sorun olmaya devam etmektedir. Önerilen sistemin bu alandaki katkısı yapısal kısıtlama ilkesine dayanmaktadır: belge modunda çalışan BDM, hiçbir koşulda parametrik bilgisini yanıtı katamaz; benzerlik eşiği kapısı aşılamadığında sistem "Bu belge bu konuyu kapsamamaktadır" yanıtını deterministik olarak döndürür. Bu yaklaşım, Li ve ark.'nın (2025) önerdiği HalluGuard [20] ve Ding ve ark.'nın (2025) ROWEN'ından [16] ayrılmaktadır: söz konusu çalışmalar halüsinasyonu model çıktısı sonrasında tespit ederken, önerilen sistem bunu istem düzeyinde yapısal kısıtlamayla önceden engellemektedir.

2.6. Mevcut Yaklaşımların Karşılaştırmalı Değerlendirmesi

İncelenen çalışmalar ışığında, bilgi erişim ve üretim sistemlerinin beş temel boyutta değerlendirilmesi mümkündür: mimari yaklaşım, veritabanı desteği, halüsinasyon riski, yapısal sorgu yeteneği ve belge geri alma kapasitesi.

Saf BDM yaklaşımları yüksek halüsinasyon riski taşımakta ve canlı veritabanlarına erişememektedir. TF-IDF/BM25 tabanlı sistemler yapısal sorgu üretememektedir. İnce ayarlı modeller alana özgü yüksek başarımlar sunsa da veritabanı desteğinden yoksundur. Genel RAG sistemleri belge sorgulama konusunda güçlü olmakla birlikte SQL üretememektedir. Metin-SQL sistemleri yapısal sorgulama açısından güçlüyken belge modunu desteklememektedir.

Önerilen sistem bu tabloda tek istisnadır: niyet yönlendirmeli mimarisi sayesinde hem SQL hem de belge sorgulama yeteneğini bir arada sunmakta, halüsinasyon riskini yapısal kısıtlama ile minimize etmekte ve web araması entegrasyonu ile güncel bilgi erişimini de kapsamına almaktadır. Bu bütünleşik yaklaşım, özellikle tek bir konuşma akışında heterojen bilgi kaynaklarına ihtiyaç duyulan akademik ortamlar için belirgin bir avantaj oluşturmaktadır.

BÖLÜM 3. SİSTEM MİMARİSİ

Mühendislik projelerinde tasarım aşaması; belirlenen problemlerin çözüme kavuşturulması için teorik bilgilerin, teknik gereksinimlerin ve kısıtların somut bir mimariye dönüştürüldüğü en kritik süreçtir. Bu bölümde, "Eğitim Alanında Özelleştirilmiş RAG Tabanlı Akademik Chatbot" projesinin sistem mimarisi, bileşen tasarımları, veri akış mekanizmaları ve bileşenler arası haberleşme protokolleri ayrıntılı biçimde açıklanmaktadır.

Sistem tasarımındaki temel felsefe üç ilke üzerine kuruludur. Birincisi modülerlik: her bileşen bağımsız olarak geliştirilebilmeli, test edilebilmeli ve değiştirilebilmelidir. İkincisi sorumlulukların ayrılığı (Separation of Concerns): niyet sınıflandırma, SQL üretimi, belge yönetimi ve web araması birbirinden izole modüller hâlinde tasarlanmıştır. Üçüncüsü savunmacı tasarım: sistem, yanlış yanıt üretmek yerine bilgi sınırlarını açıkça belirtmeyi tercih etmektedir.

3.1. Genel Mimari ve Katmanlı Yapı

Geliştirilen sistem, yazılım mühendisliğinde yaygın olarak benimsenen N-Katmanlı (N-Tier) mimari prensibi doğrultusunda dört temel katman üzerine inşa edilmiştir.

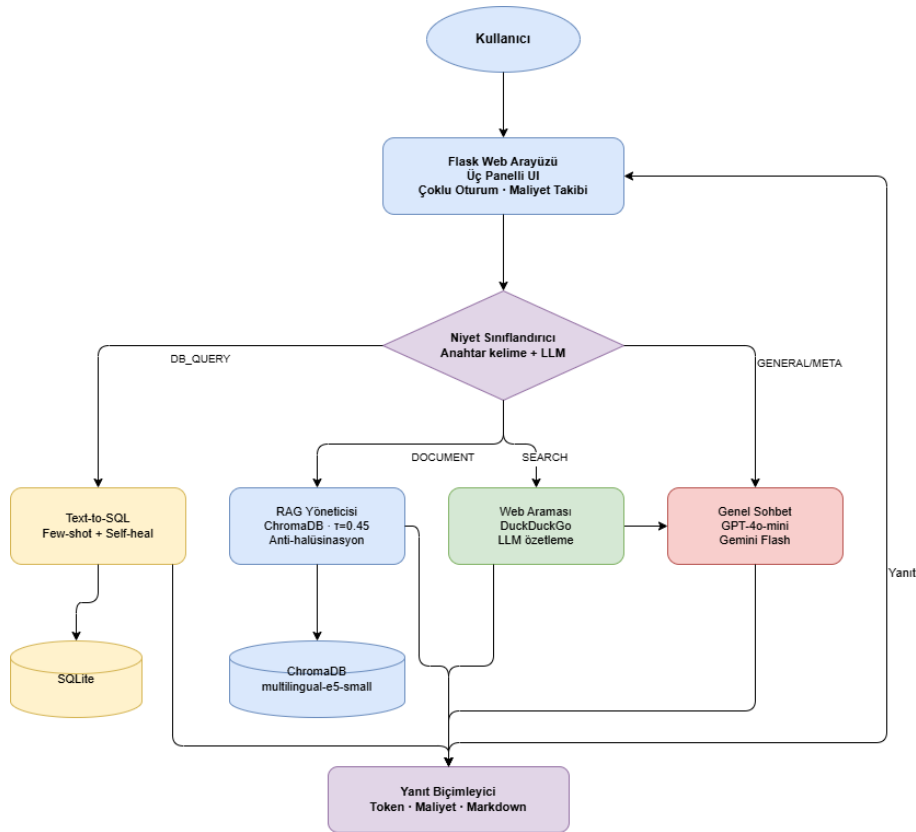
Sunum katmanı, kullanıcının sistemle doğrudan etkileşime girdiği en üst katmandır. Saf HTML, CSS ve JavaScript ile geliştirilen üç panelli web arayüzü bu katmanı oluşturmaktadır. Flask'ın sunduğu statik dosya servisi aracılığıyla istemciye iletilmekte; REST API çağrıları ise JavaScript'in fetch arayüzüyle asenkron olarak gerçekleştirilmektedir.

Uygulama katmanı, sistemin iş mantığını barındıran ve tüm yapay zekâ orkestrasyonunu yöneten merkezi katmandır. Flask web çerçevesi üzerine inşa edilmiş RESTful API endpointleri, bu katmanın iskeletini oluşturmaktadır. Niyet

sınıflandırıcı, Text-to-SQL modülü, RAG yöneticisi ve web araması bileşenleri bu katmanda yer almakta; LangChain orkestrasyon çerçevesi tüm bileşenleri koordine etmektedir.

Yapay zekâ ve çıkarım katmanı, büyük dil modellerine erişimi soyutlayan katmandır. `llm_olustur()` fabrika fonksiyonu aracılığıyla GPT-4o-mini (OpenAI API) veya Gemini Flash (Google Generative AI API) modellerinden biri çalışma zamanında seçilmektedir. Bu soyutlama sayesinde model değişikliği, üst katmandaki iş mantığında herhangi bir değişiklik gerektirmemektedir.

Veri katmanı, kalıcı depolama sorumluluğunu üstlenen en alt katmandır. İlişkisel veriler SQLite veritabanında SQLAlchemy aracılığıyla yönetilmektedir. Vektörel veriler ise ChromaDB'nin kalıcı depolama modunda `./chroma_db/` dizini altında tutulmaktadır. Oturum durumu (conversation state) uygulama belleğinde conversations sözlüğünde saklanmaktadır.



Şekil 3.1. Sistem Mimarisi Blok Şeması

3.2. Yazılım Gereksinim Listesi

Sistemin geliştirilmesi sürecinde fonksiyonel ve fonksiyonel olmayan gereksinimler belirlenmiş; bu gereksinimler tasarım kararlarının temelini oluşturmuştur.

Fonksiyonel gereksinimler şunlardır: sistem doğal dil sorgularını SQL'e dönüştürerek ilgili veritabanı tablosunu sorgulayabilmelidir; sistem yüklenen PDF ve Excel belgelerinden bağlama duyarlı bilgi geri alabilmelidir; sistem her kullanıcı ifadesini beş kategoriden birine sınıflandırarak uygun işlem hattına yönlendirmelidir; sistem belge kapsamı dışındaki sorgulara açıkça ret yanıtı üretmeli ve asla parametrik bilgisiyle yanıt tamamlamamalıdır; sistem gerçek zamanlı internet araması yaparak güncel bilgi döndürebilmelidir; sistem GPT-4o-mini ve Gemini Flash modellerini çalışma zamanında değiştirilebilir biçimde desteklemelidir; sistem her konuşma oturumu için token tüketimini ve tahmini USD maliyetini anlık olarak görüntüleyebilmelidir; sistem birden fazla belgeyi eş zamanlı olarak yönetebilmeli, belge düzeyinde silme işlemini desteklemelidir; sistem birden fazla paralel sohbet oturumunu bağımsız geçmişlerle yönetebilmelidir.

Fonksiyonel olmayan gereksinimler ise şunlardır: ortalama sorgu yanıt süresi 5 saniyenin altında kalmalıdır; embedding modeli çevrimdışı çalışabilmelidir ve harici bir API'ya bağımlı olmamalıdır; API anahtarları kaynak koda gömülmemeli, ortam değişkenleri aracılığıyla yönetilmelidir; veritabanı sorguları SQL enjeksiyon saldırılarına karşı parametreliliğiyle korunmalıdır; sistem Python 3.11 ortamında, ek yapılandırma gerektirmeksizin tek komutla başlatılabilmelidir; sistem modüler yapısıyla yeni BDM sağlayıcıları veya veritabanı şemaları için mevcut kodu değiştirmeksizin genişletilebilir olmalıdır.

3.3. Birimler Arası Bilgi Akışı

Sistemdeki veri akışı belirli bir protokol zinciri izlemektedir. Kullanıcı mesajı sunum katmanından POST /api/chat endpointine JSON formatında iletilir; bu istek {"message": "...", "model": "chatgpt", "conversation_id": "..."} yapısını taşır. Uygulama katmanı mesajı alarak önce oturum bağlamını hazırlar, ardından niyet

sınıflandırıcıya iletir. Sınıflandırıcı niyet kategorisini döndürdükten sonra ilgili işlem hattı çağrılır. İşlem hattının çıktısı yanıt biçimleyiciden geçirilerek {"response": "...", "tokens": 1240, "cost": 0.000372} yapısında JSON olarak istemciye döndürülür. İstemci bu yanıtı ayrıştırarak sohbet arayüzünü, token sayacını ve maliyet göstergesini günceller.

Bileşenler arası haberleşmede Python nesne referansları kullanılmakta; rag_manager, db ve llm_default nesneleri modül düzeyinde başlatılarak tüm işlev çağrılarında paylaşılmaktadır. Bu yaklaşım nesne oluşturma maliyetini ortadan kaldırmakla birlikte çok iş parçacıklı (multi-thread) ortamda durum tutarlılığının dikkatle yönetilmesini gerektirmektedir.

3.4. Niyet Sınıflandırma ve Yönlendirme Motoru

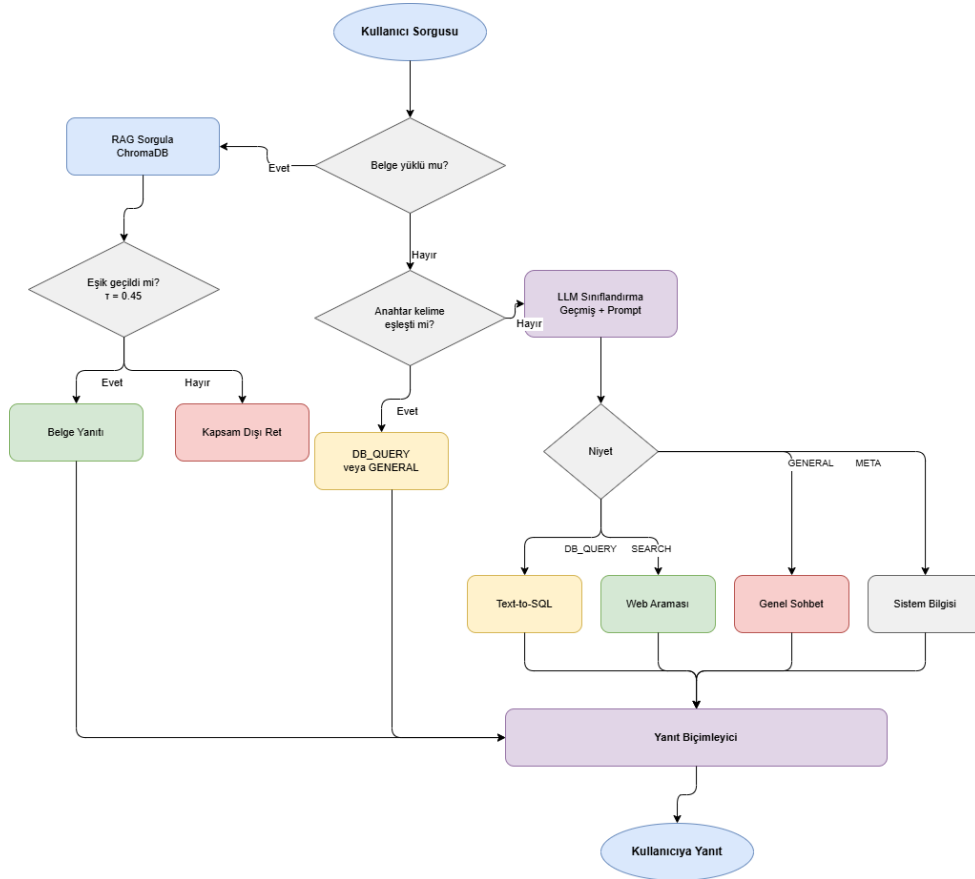
3.4.1. İki Kademeli Sınıflandırma Stratejisi

Niyet sınıflandırıcısı, her kullanıcı ifadesini ve konuşma geçmişini alarak beş kategoriden birine atayan merkezi karar birimidir: DB_QUERY akademik veritabanı sorguları için, DOCUMENT yüklenen belgeler üzerindeki sorular için, SEARCH internet araması gerektiren güncel bilgi talepleri için, GENERAL selamlama ve genel konuşma için, META sistemin kendi durumuna ilişkin sorgular için kullanılmaktadır.

Birinci kademe anahtar kelime tabanlı hızlı ön filtrelemedir. Bu kademe, yüksek güvenilirlikli ve sık tekrarlanan sorgu türlerinde gereksiz LLM çağrısından kaçınarak sistem gecikmesini ve API maliyetini azaltmaktadır. "öğrenci", "ders", "not", "vize", "final", "hoca", "bölüm", "akademisyen" gibi veritabanı varlık terimleri DB_QUERY'ye; "selam", "merhaba", "nasılsın", "teşekkür" gibi sosyal ifadeler ise GENERAL'e yönlendirilmektedir.

İkinci kademe BDM tabanlı sınıflandırmadır. Anahtar kelime eşleştirmesiyle çözülemeyen belirsiz sorgular için devreye giren bu kademe, yapılandırılmış bir sistem yönergesi ve son beş mesajdan oluşan konuşma geçmişini BDM'ye iletir;

modelden yalnızca tek bir kategori kelimesi döndürmesi istenir. Bu tasarım hem token verimliliğini artırmakta hem de sınıflandırma çıktısının ayrıştırılmasını kolaylaştırmaktadır.



Şekil 3.2 Niyet Yönlendirme Akış Şeması

3.4.2. Belge Yüklü Durum Önceliği

Sistemde bir veya daha fazla belge yüklü olduğunda, niyet sınıflandırmasından önce RAG sorgulaması öncelikli olarak denenmektedir. Bu tasarım kararının gerekçesi şudur: kullanıcı bir belge yüklediğinde, sonraki sorularının büyük bölümünü o belgeyle ilişkili olarak yöneltmesi beklenmektedir. Bu nedenle META dışındaki tüm niyetler önce RAG Manager'a yönlendirilmekte; yeterli benzerlik skoru elde edilemezse ilgili işlem hattına devredilmektedir.

3.5. Few-Shot Text-to-SQL Modülü

3.5.1.Şema Enjeksiyonu

SQL üretim sürecinin ilk adımı, veritabanı şemasının tamamının istem bağlamına eklenmesidir. LangChain'in `SQLDatabase.get_table_info()` yöntemi aracılığıyla tüm tablo tanımları (CREATE TABLE ifadeleri), sütun adları ve veri tipleri tek seferlik alınarak önbelleğe alınmaktadır. Bu bilgi her SQL üretim istemine eklenerek BDM'nin alan adlandırma kurallarından ve tablo ilişkilerinden tam olarak haberdar olması sağlanmaktadır.

3.5.2.Anlamsal Few-Shot Örnek Seçimi

Sisteme dahil edilen yedi adet el yapımı soru-SQL çifti, farklı sorgu karmaşıklık düzeylerini temsil edecek biçimde seçilmiştir: tek tablolü COUNT sorguları, JOIN içeren birleşim sorguları, LIKE ile filtreleme ve çok tablolü toplama sorguları bu kategorilerin başında gelmektedir. Her sorgu için LangChain'in `SemanticSimilarityExampleSelector` bileşeni, `intfloat/multilingual-e5-small` gömme modeli ve ChromaDB üzerinde kosinüs benzerliği hesaplayarak $k=3$ en yakın örneği dinamik olarak seçmektedir. Sabit bir örnek kümesi yerine anlamsal seçim kullanmak, basit sorguların tek tablolü örneklerle, karmaşık sorguların ise çok tablolü birleşim örnekleriyle eşleşmesini güvence altına almaktadır.

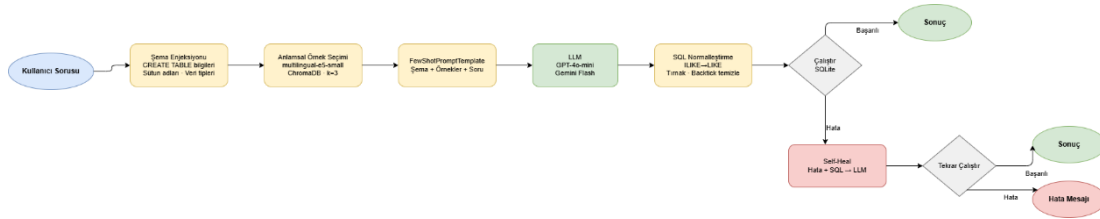
3.5.3.SQL Üretimi ve Normalleştirme

`FewShotPromptTemplate` şema bilgisi, seçilen üç örnek ve kullanıcı sorgusunu birleştirerek BDM'ye iletilmektedir. BDM çıktısı üzerinde aşağıdaki normalleştirme adımları uygulanmaktadır: ters tırnak (backtick) temizleme, `ILIKE` → `LIKE` dönüşümü (SQLite'in `ILIKE`'i desteklememesi nedeniyle), çift tırnak → tek tırnak dönüşümü ve sorgu metninin başındaki açıklama satırlarının ayıklanması.

3.5.4.Öz-Onarımlı Düzeltme Döngüsü

Normalleştirilmiş SQL, `SQLAlchemy` aracılığıyla veritabanına gönderilmekte ve bir istisna fırlatılırsa öz-onarım mekanizması devreye girmektedir. Hata mesajı ve özgün

SQL sorgusu yeni bir istemde BDM'ye beslenmekte; modelden yalnızca düzeltilmiş SQL döndürmesi istenmektedir. Sistem tek bir düzeltme denemesi yapmaktadır bu, çok adımlı yeniden denemelerden kaynaklanabilecek sonsuz döngü riskini ortadan kaldırmaktadır. Değerlendirme sonuçları bu mekanizmanın yürütme hatalarının %80'ini giderdiğini ortaya koymaktadır.



Şekil 3.3 Few-Shot SQL Pipeline

3.6. RAG Belge Yöneticisi ve Halüsinasyon Önleme Politikası

3.6.1. Belge İşleme Hattı

RagManager sınıfı, belge yükleme, parçalama, gömme ve sorgulama işlemlerini koordine eden merkezi bileşendir. PDF dosyaları PyPDFLoader, Excel dosyaları pandas ile metin dönüşümü ardından TextLoader ile yüklenmektedir. RecursiveCharacterTextSplitter, her belgeyi 1000 karakterlik, 200 karakterlik örtüşmeli parçalara bölmektedir. Bu örtüşme, parça sınırlarındaki bağlam kopukluklarını en aza indirmektedir.

Her belge için ./chroma_db/{belge_adı}/ dizini altında ayrı bir ChromaDB koleksiyonu oluşturulmaktadır. Bu belge başına izolasyon yaklaşımı iki önemli avantaj sağlamaktadır: birincisi, bir belgenin silinmesi diğer belgelerin gömme verilerini etkilemez; ikincisi, çoklu belge sorgularında kaynaklar arası çakışma yaşanmaz.

3.6.2. Dinamik Chunk Bütçelemesi

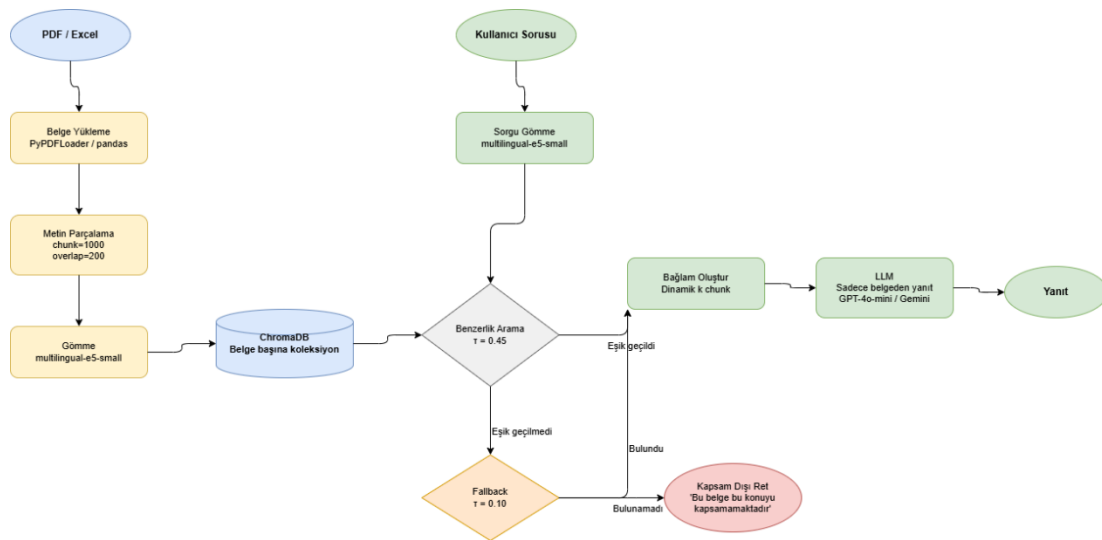
Birden fazla belge yüklü olduğunda bağlam penceresi yönetimi kritik bir mühendislik problemi hâline gelmektedir. Sistem bu sorunu belge başına dinamik k

hesaplamasıyla çözmektedir: $k_{\text{belge}} = \max(1, \lfloor K_{\text{max}} / n_{\text{belge}} \rfloor)$, burada $K_{\text{max}} = 20$ 'dir. Bu formül, beş belge yüklü olduğunda her belgeden en fazla dört chunk alınmasını güvence altına alarak toplam bağlam boyutunu kontrol altında tutar. Tek belge durumunda ise tüm bütçe ($k=20$) o belgeye tahsis edilir.

3.6.3. Eşik Kapısı ve Sıfır-Halüsinasyon Politikası

Sorgulama aşamasında birincil benzerlik eşiği $\tau=0.45$ uygulanmaktadır. Bu eşiğin belirlenmesinde demo belge koleksiyonu üzerinde ampirik testler yapılmış; düşük değerlerde kapsam dışı parçaların yanıtla karışması, yüksek değerlerde ise ilgili parçaların elenmesi gözlemlenmiştir. $\tau=0.45$, bu iki risk arasındaki en iyi dengeyi sunmaktadır.

Hiçbir parça birincil eşiği aşamazsa $\tau=0.10$ ile geniş kapsamlı yedek (fallback) tarama yapılmaktadır. Bu tarama da sonuçsuz kalırsa sistem deterministik olarak "Bu belge bu konuyu kapsamamaktadır" yanıtını döndürmekte ve BDM'ye hiçbir üretim yetkisi tanımamaktadır. Bu politika, model öz-denetimine değil yapısal kısıtlamaya dayanan bir halüsinasyon önleme yaklaşımı olarak tasarlanmıştır.



Şekil 3.4 RAG Pipeline

3.7. Web Araması Entegrasyonu

SEARCH niyeti tespit edildiğinde sistem, LangChain'in DuckDuckGoSearchRun aracını çağırarak kullanıcı sorgusunu internet arama motoruna iletmektedir. Dönen arama sonuçları, kaynakları belirterek özetlemesi için BDM'ye aktarılmakta ve atıflandırılmış bir yanıt üretilmektedir. DuckDuckGo'nun seçilmesinin başlıca nedeni gizlilik odaklı yapısı ve üçüncü taraf API anahtarı gerektirmemesidir. Hız sınırlaması ve arama kalitesi değişkenliği bu bileşenin bilinen kısıtları arasında yer almaktadır; hız sınırı aşıldığında sistem doğrudan BDM yanıtına düşmekte ve kullanıcıyı bilgilendirmektedir.

3.8. Token ve Maliyet Takip Mekanizması

Her `llm.invoke()` çağrısını saran `llm_invoke_tracked()` sarmalayıcı işlevi, token kullanımını `usage_metadata` (LangChain v0.2+) veya `response_metadata.token_usage` (eski OpenAI yedek yolu) üzerinden okumaktadır. Maliyet hesaplama formülü şu şekilde uygulanmaktadır: $\text{maliyet} = (\text{token} / 1.000.000) \times r_{\text{model}}$. GPT-4o-mini için $r = 0,30$ USD/milyon token, GPT-4o için $r = 3,75$ USD/milyon token ve Gemini Flash için $r = 0,075$ USD/milyon token değerleri kullanılmaktadır. Her çağrı sonrasında token sayısı ve maliyet hem global toplama hem de aktif konuşmanın bireysel sayacına eklenmektedir.

3.9. Çoklu Oturum Yönetimi

Sistem, bağımsız geçmişlere ve maliyet sayaçlarına sahip birden fazla paralel konuşma oturumunu desteklemektedir. `conversations` sözlüğünde saklanan her oturum şu alanları içermektedir: benzersiz kimlik (`conv_id`), oturumun ilk mesajından BDM tarafından üretilen kısa başlık (`name`), son beş mesajla sınırlı tutulan geçmiş (`history`), oturuma ait kümülatif token sayısı (`tokens`) ve oturuma ait kümülatif USD maliyeti (`cost`). Geçmiş uzunluğunun beş mesajla sınırlandırılması, bağlam penceresi taşmasını engelleyen önemli bir tasarım kararıdır.

RAG Karşılaştırma Modu etkinleştirildiğinde sistem, aynı sorgu için hem RAG destekli hem de saf BDM yanıtını paralel olarak üretmektedir. Bu özellik belge temellenen yanıt kalitesine katkısını yan yana görselleştirerek deneysel değerlendirmeye olanak tanımaktadır.

3.10. Kullanılan Teknolojiler ve Araçlar

Sistemin teknoloji yığını seçiminde üç temel kriter belirleyici olmuştur: Python ekosistemine uyum, açık kaynak tercih ve düşük operasyonel maliyet. Çekirdek bağımlılıklar şu şekilde özetlenebilir: web çerçevesi olarak Flask 3.0, LLM orkestrasyonu için LangChain 0.2+, vektör veritabanı olarak ChromaDB (kalıcı mod), gömme modeli olarak intfloat/multilingual-e5-small (HuggingFace), ilişkisel veritabanı yönetimi için SQLite ve SQLAlchemy, belge yükleme için PyPDFLoader ve pandas, web araması için DuckDuckGo Search, BDM sağlayıcıları olarak OpenAI GPT-4o-mini ve Google Gemini Flash.

Bu teknoloji tercihlerinin en dikkat çekici özelliği, gömme modelinin çevrimdışı çalışabilmesidir. İnternet bağlantısı olmayan kurumsal ortamlarda bile belge RAG işlevselliği tam olarak kullanılabilir. Yalnızca BDM API çağrıları ve web araması işlevleri internet bağlantısı gerektirmektedir.

BÖLÜM 4. UYGULAMA

Mühendislik projelerinde tasarım süreci ne kadar titizlikle planlanmış olursa olsun, projenin nihai başarısı bu planın çalışır, kararlı ve test edilebilir bir sisteme dönüştürülmesiyle ölçülür. Bu bölümde; önceki bölümlerde teorik altyapısı, sistem mimarisi ve bileşen tasarımları sunulan ajansal akademik chatbot projesinin fiziksel gerçekleştirme aşamaları ayrıntılı biçimde ele alınmaktadır.

Uygulama süreci, yazılım geliştirme yaşam döngüsüne (SDLC) uygun olarak üç ana ekseninde yürütülmüştür. Birinci aşamada yapay zekâ bileşenlerinin gömme modeli, vektör veritabanı ve BDM sağlayıcıları kurulumu ve entegrasyonu tamamlanmıştır. İkinci aşamada iş mantığını yöneten Flask API servisleri ile veritabanı bağlantıları hayata geçirilmiştir. Üçüncü ve son aşamada ise kullanıcının sistemle doğrudan etkileşime girdiği web arayüzü kodlanmıştır. Bu bölüm kapsamında; geliştirme ortamı, veritabanı kurulumu, gömme altyapısı, API mimarisi, kullanıcı arayüzü tasarımı, web tarama bileşeni ve karşılaşılan teknik darboğazlar ile bu sorunlara getirilen mühendislik çözümleri somut verilerle aktarılmaktadır.

4.1. Geliştirme Ortamı ve Kurulum

4.1.1. Donanım ve Yazılım Altyapısı

Sistem, birincil geliştirme ortamı olarak Google Colab T4 GPU altyapısı üzerinde oluşturulmuştur. GPU hızlandırması gömme modeli için isteğe bağlı olmakla birlikte, tüm bileşenler yalnızca CPU ile de tam işlevsellik sunacak şekilde tasarlanmıştır. Yerel geliştirme ortamında Python 3.11 kullanılmış; bağımlılıklar requirements.txt dosyasında sabitlenmiştir.

Temel Python paketleri ve sürümleri şu şekilde belirlenmiştir: flask==3.0.x web çerçevesi, langchain==0.2.x ile langchain-community, langchain-openai, langchain-google-genai, langchain-chroma, langchain-huggingface ve langchain-text-splitters

alt paketleri LLM orkestrasyonu için, chromadb vektör veritabanı için, sqlalchemy ilişkisel veritabanı yönetimi için, pypdf PDF işleme için, pandas ve openpyxl Excel desteği için, duckduckgo-search web araması için kullanılmaktadır.

4.1.2. Bağımlılık Başlatma Stratejisi

Uygulamanın başlatma süresini kısaltmak amacıyla ertelenmiş (lazy) başlatma yaklaşımı benimsenmiştir. `ensure_imports()` işlevi, ağır bağımlılıkların (LangChain, ChromaDB, gömme modeli) yalnızca ilk HTTP isteği geldiğinde yüklenmesini sağlamaktadır. Bu sayede Flask sunucusu birkaç saniyede ayağa kalkmakta; BDM bağlantıları yalnızca gerçek kullanım sırasında kurulmaktadır. API anahtarları `os.environ` üzerinden okunmakta, kaynak koda hiçbir şekilde gömülmemektedir.

4.2. Veritabanı Kurulumu ve Şema Tasarımı

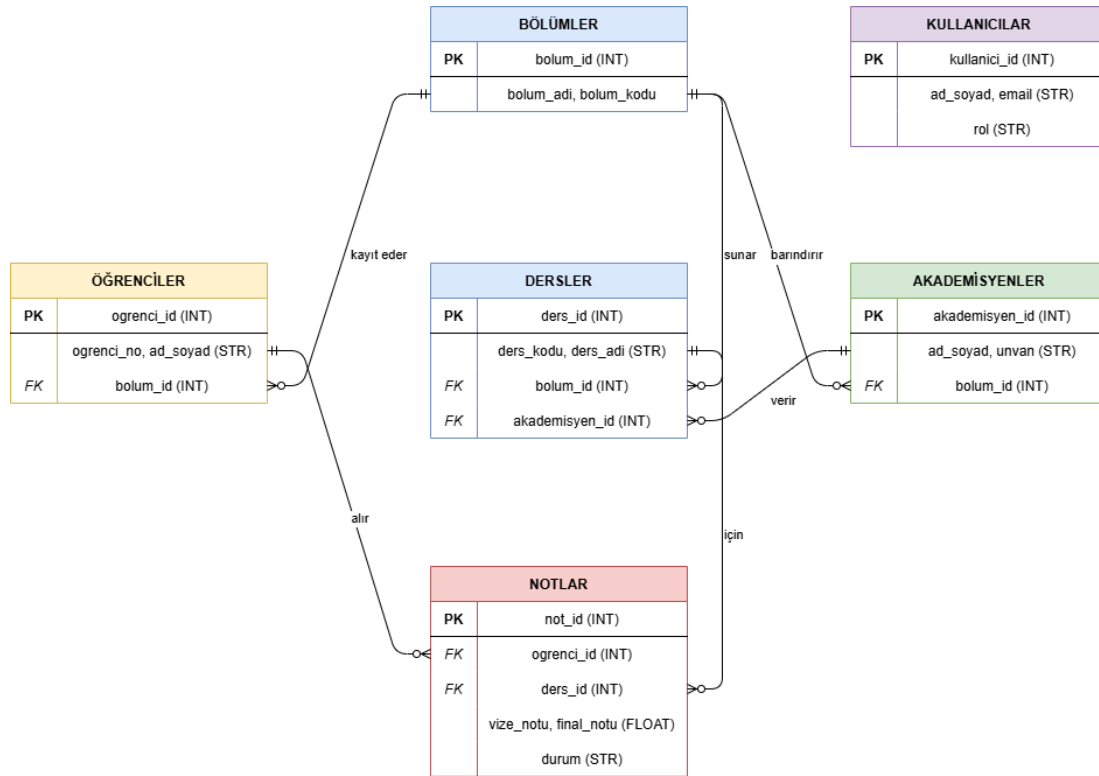
4.2.1. Demo Veritabanının Oluşturulması

Demo SQLite veritabanı (`demo_okul.db`), uygulama ilk kez çalıştırıldığında `init_db()` işlevi tarafından otomatik olarak oluşturulmaktadır. Bu yaklaşım, kullanıcının herhangi bir veritabanı yönetim aracına ihtiyaç duymaksızın sistemi çalıştırmasını sağlamaktadır. SQLAlchemy'nin `create_engine()` ve `text()` arayüzleri tüm veritabanı işlemlerinde kullanılmaktadır.

Veritabanı altı tablodan oluşmaktadır ve Sakarya Üniversitesi Bilgisayar Mühendisliği bölümünün gerçek yapısını yansıtacak biçimde modellenmiştir. Tablo başına kayıt sayıları şu şekildedir:

Tablo 4.1. Demo Veri Tabanı Şeması ve Kayıt Sayıları

Tablo	Kayıt Sayısı	Açıklama
kullanıcılar	2	Sistem kullanıcıları
bolumler	5	Akademik bölümler
akademisyenler	8	Öğretim üyeleri
ogrenciler	15	Kayıtlı öğrenciler
dersler	10	Aktif dersler
notlar	35	Sınav notları



Şekil 4.1 ER Diyagramı

4.2.2. Şema Enjeksiyon Mekanizması

SQL üretiminde BDM'nin tablo yapısını eksiksiz bilmesi kritik öneme sahiptir. `SQLDatabase.get_table_info()` yöntemiyle elde edilen şema bilgisi her tablonun `CREATE TABLE` ifadesi, sütun adları ve veri tipleri her SQL üretim istemine eklenmektedir. Bu bilgi uygulama başlangıcında tek seferlik alınarak önbellege

alınmaktadır; böylece her sorguda tekrarlayan veritabanı çağrısından kaçınılmaktadır.

4.2.3. SQL Güvenliği

Veritabanı sorguları hiçbir koşulda doğrudan dize birleştirme yöntemiyle oluşturulmamaktadır. SQLAlchemy'nin `text()` ve `execute()` arayüzleri parametrelili sorgu yapısını zorunlu kılmaktadır. Bu yaklaşım, BDM tarafından üretilen SQL'in bile doğrudan çalıştırılmadan önce sözdizimi doğrulamasından geçirilmesini sağlamakta ve olası SQL enjeksiyon saldırılarını yapısal olarak engellemektedir.

4.3. Gömme Modeli ve Vektör Veritabanı

4.3.1. Gömme Modeli Seçimi ve Gerekçesi

`intfloat/multilingual-e5-small` modeli birçok kriterin bir arada değerlendirilmesiyle seçilmiştir. Model boyutu yaklaşık 120 MB olup CPU üzerinde kabul edilebilir yanıt süreleri sağlamaktadır. Çok dilli destek sayesinde Türkçe ve İngilizce karma sorgu-belge çiftlerinde tutarlı anlamsal eşleştirme yapabilmektedir. HuggingFace üzerinden `langchain-huggingface` entegrasyonu ile sorunsuz kullanılabilmekte ve internet bağlantısı olmaksızın çevrimdışı modda çalışabilmektedir.

Model; hem few-shot SQL örnek seçiminde hem de RAG belge parça geri alımında aynı vektör uzayı üzerinde çalışmaktadır. Bu tutarlılık, farklı görevler için farklı gömme modellerine ihtiyaç duyulmamasını sağlamaktadır.

4.3.2. ChromaDB Koleksiyon Yönetimi

Her yüklenen belge için `./chroma_db/{belge_adı}/` dizini altında ayrı ve kalıcı bir ChromaDB koleksiyonu oluşturulmaktadır. Bu belge başına izolasyon yaklaşımı üç önemli işlevsel avantaj sağlamaktadır: birincisi, bir belgenin silinmesi diğer belgelerin gömme verilerini etkilemez; ikincisi, çoklu belge sorgularında kaynaklar arası vektör karışması yaşanmaz; üçüncüsü, uygulama yeniden başlatıldığında

mevcut koleksiyonlar kalıcı diskten otomatik olarak yüklenir ve yeniden indeksleme gerekmez.

similarity_score_threshold yöntemiyle gerçekleştirilen benzerlik araması, eşik değerinin altında kalan parçaların sonuç kümesine dahil edilmesini engellemektedir. Bu mekanizma, düşük kaliteli eşleşmelerin yanıtta karışmasını önleyen temel güvenlik katmanını oluşturmaktadır.

4.4. Flask API Mimarisi

4.4.1. RESTful Endpoint Tasarımı

Flask uygulaması host=0.0.0.0 ve port=5000 ile başlatılmaktadır. Tüm API endpointleri JSON üzerinden haberleşmekte ve Content-Type: application/json başlığı zorunlu tutulmaktadır. Temel endpointler şunlardır:

POST /api/chat Sohbet mesajı gönderme. İstek gövdesi message, model ve conversation_id alanlarını içermektedir. Yanıt olarak response, tokens ve cost döndürülmektedir. Bu endpoint tüm niyet yönlendirme ve işlem hattı mantığını kapsamaktadır.

POST /api/documents/upload Belge yükleme. Multipart form verisiyle PDF veya Excel dosyası kabul etmektedir. Yüklenen dosya önce uploads/ dizinine kaydedilmekte, ardından RagManager.add_document() çağrısıyla ChromaDB'ye eklenmektedir.

DELETE /api/documents/{belge_adi} Belge silme. İlgili ChromaDB koleksiyonunu ve disk dosyasını birlikte temizlemektedir.

GET /api/documents Yüklü belgelerin listesini döndürmektedir.

POST /api/conversations Yeni sohbet oturumu oluşturmaktadır. Benzersiz conv_id üreterek conversations sözlüğüne ekler.

GET /api/conversations Tüm aktif oturumların adlarını ve maliyetlerini listeler.

DELETE /api/conversations/{conv_id} Oturum silme.

POST /api/crawl Web sitesi tarama başlatma. URL ve maksimum sayfa sayısını alarak arka planda tarayıcıyı çalıştırır.

4.4.2. Oturum Durumu Yönetimi

Konuşma durumu sunucu belleğinde conversations sözlüğünde tutulmaktadır. Her oturum nesnesi şu alanları içermektedir: name (ilk mesajdan BDM aracılığıyla üretilen 4-5 kelimelik Türkçe başlık), history (son beş mesajla sınırlı liste), tokens (kümülatif token sayısı) ve cost (kümülatif USD maliyet). Oturum başlığı ilk mesaj gönderildiğinde conv_isim_uret() işleviyle asenkron üretilmekte; bu sayede kullanıcı arayüzünde anlamlı bir oturum adı görüntülenmektedir.

4.5. Kullanıcı Arayüzü Tasarımı

4.5.1. Üç Panelli Yerleşim Mimarisi

Web arayüzü saf HTML5, CSS3 ve JavaScript ile geliştirilmiş; herhangi bir ön uç çerçevesine bağımlılıktan kaçınılmıştır. Bu tercih, bağımlılık güncellemelerinden kaynaklanan kırılganlığı ortadan kaldırmakta ve uygulamayı taşınabilir kılmaktadır.

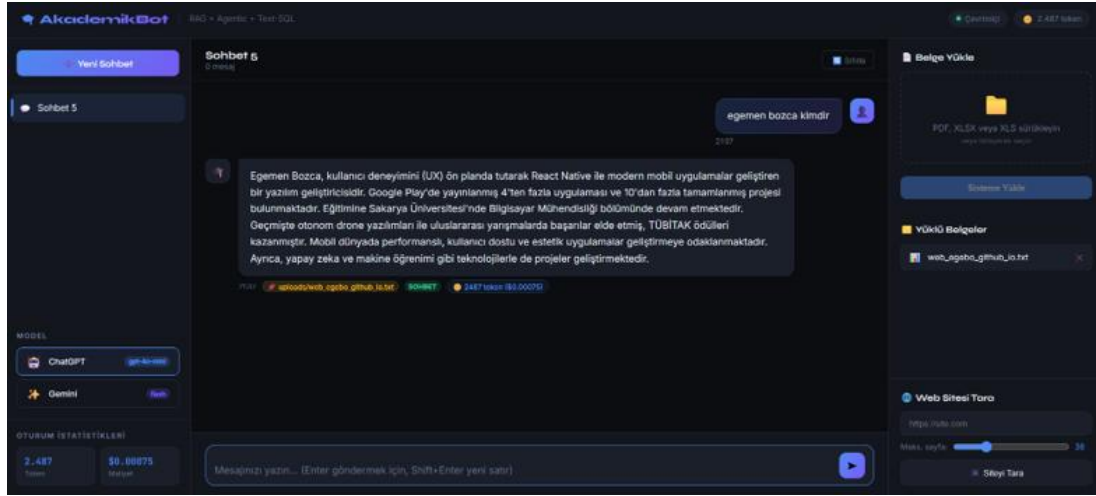
Arayüz, CSS Grid ve Flexbox yapıları kullanılarak üç dikey panele bölünmüştür.

Sol panel model ve oturum yönetimini kapsamaktadır. Bu panelde GPT-4o-mini ve Gemini Flash arasında geçiş yapan model seçici, yeni sohbet oluşturma düğmesi, mevcut oturumları listeleyen tıklanabilir oturum listesi ve aktif oturumun token ile USD maliyet göstergesi bulunmaktadır.

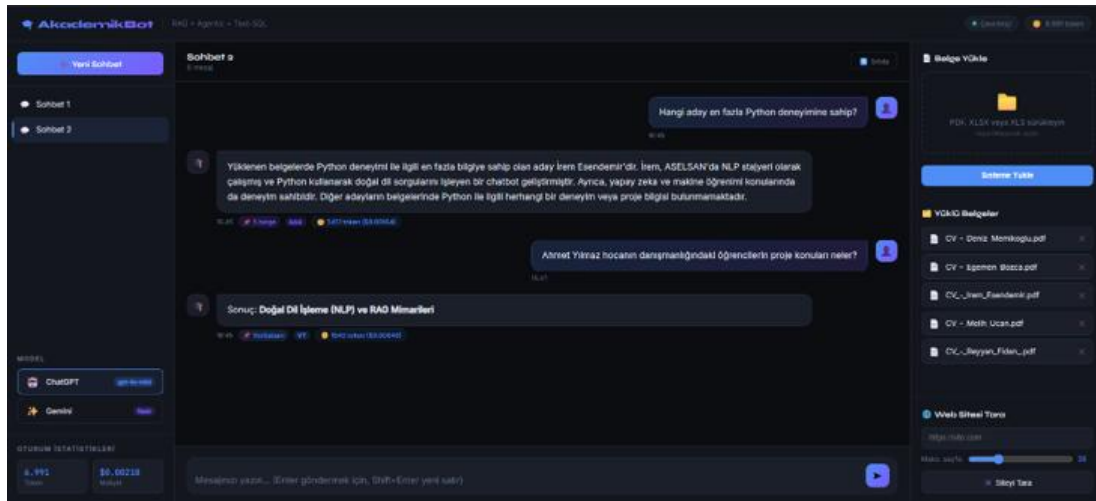
Orta panel kullanıcının birincil etkileşim alanıdır. Sohbet geçmişini yukarıdan aşağıya listeleyen mesajlaşma ekranı, alt kısımda metin girdi kutusu ve gönder düğmesi ile oturumu temizleyen sıfırlama düğmesinden oluşmaktadır. Kullanıcı

mesajları sağa, sistem yanıtları sola hizalanmış baloncuklar hâlinde görüntülenmektedir.

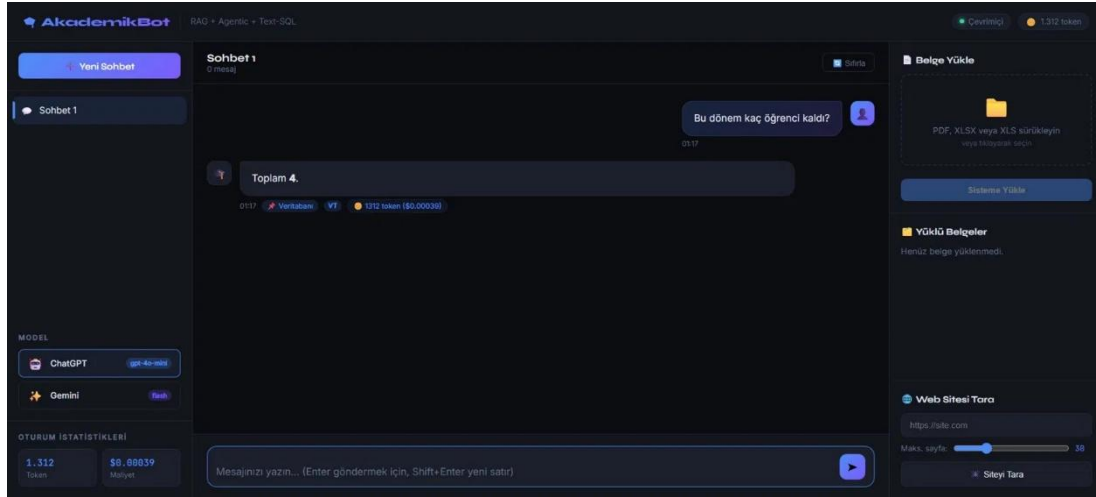
Sağ panel belge ve web yönetimini üstlenmektedir. PDF ve Excel dosyalarını kabul eden çoklu dosya yükleme bileşeni, yüklü belgelerin listesini gösteren açılır menü ve belge silme işlevi bu panelde yer almaktadır. Bunların yanı sıra URL girişi ve maksimum sayfa sayısı kontrolü içeren web sitesi tarayıcı bölümü de sağ panelde konumlandırılmıştır.



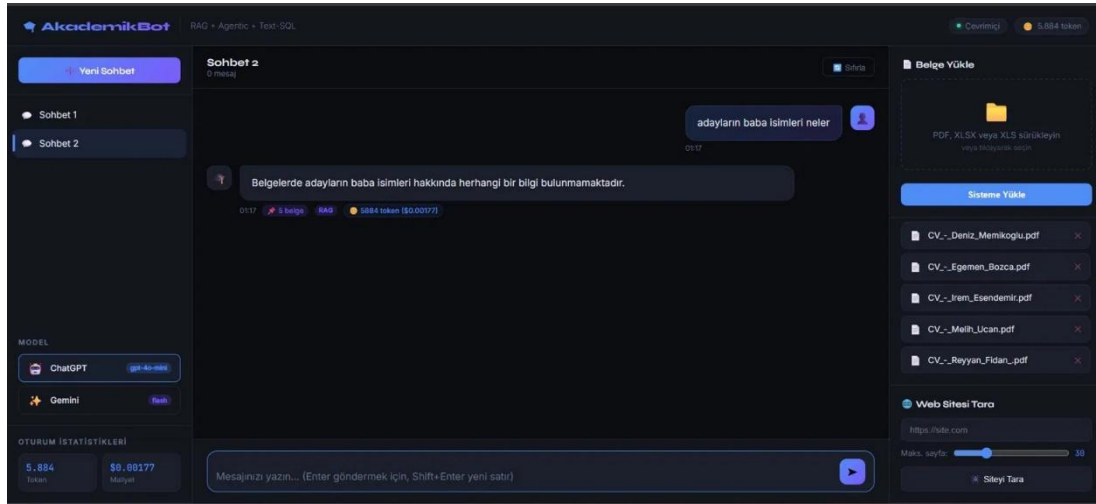
Şekil 4.2. Web Scrape Sorgusu



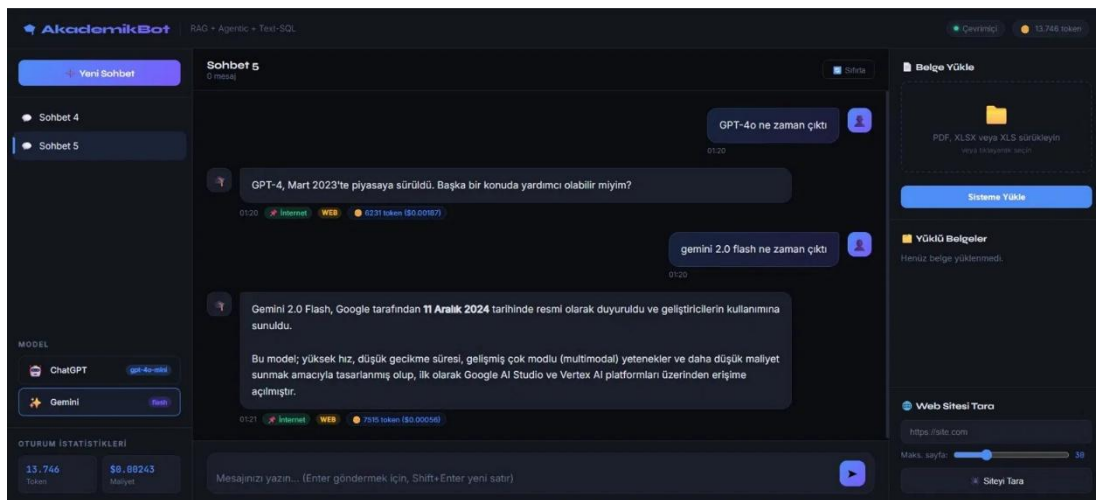
Şekil 4.3. RAG Modunda Çoklu Belge Sorgusu



Şekil 4.4. Text-to-SQL Sorgusu ve Veri Tabanı Yanıtı



Şekil 4.5. Kapsam Dışı Sorguya Ret Yanıtı



Şekil 4.6. Web Araması ve Çok Sağlayıcı Maliyet Karşılaştırması

4.5.2. Asenkron İletişim Altyapısı

Tüm API çağrıları JavaScript'in fetch arayüzüyle asenkron olarak gerçekleştirilmektedir. Kullanıcı mesajı gönderdiğinde arayüz anında görsel bir yükleme göstergesi sunmakta; yanıt geldiğinde mesaj baloncuğu güncellenmekte ve token/maliyet sayaçları yenilenmektedir. Bu yaklaşım, uzun süren BDM çağrıları sırasında arayüzün donmasını engellemektedir.

4.5.3. Markdown Render Desteği

BDM yanıtları çoğunlukla Markdown biçimlendirmesi içermektedir: başlıklar, kalın metin, maddeli listeler ve kod blokları. Arayüz bu biçimlendirmeyi doğrudan tarayıcıda HTML'ye dönüştürerek görüntülemektedir. Özellikle SQL sorguları ve kod parçacıkları `<code>` etiketiyle sabit genişlikli yazı tipinde gösterilmektedir.

4.6. Web Sitesi Tarayıcı Bileşeni

4.6.1. Tarayıcı Mimarisi

Web sitesi tarayıcısı (website_to_rag işlevi), kullanıcının belirttiği bir URL'den başlayarak belirli sayıda sayfayı otomatik olarak ziyaret eden ve içeriği RAG sistemine yükleyen bir bileşendir. urllib ve html.parser kullanılarak harici bağımlılıktan bağımsız bir uygulama tercih edilmiştir.

Tarayıcı çalışma döngüsü şu adımları izlemektedir: başlangıç URL'si ziyaret kuyruğuna eklenir; her sayfa için HTTP GET isteği gönderilir; HTML içerik BeautifulSoup ile ayrıştırılarak metin çıkarılır; aynı alan adındaki bağlantılar kuyruğa eklenir; robots.txt kurallarına uyum sağlanır; belirlenen maksimum sayfa sayısına ulaşıldığında tarama sona erer. Taranan içerik uploads/web_{alan_adi}.txt dosyasına kaydedilerek RagManager'a eklenmektedir. Bu sayede kullanıcı, herhangi bir web sitesinin içeriğini belge olarak sisteme dahil edebilmekte ve o site hakkında soru sorabilmektedir.

4.6.2. Kısıtlamalar ve Hata Yönetimi

Tarayıcı bileşeninin bilinen kısıtları şunlardır: JavaScript ile dinamik olarak oluşturulan içerikler işlenememektedir; kimlik doğrulaması gerektiren sayfalar taranamamaktadır, yoğun medya içerikli (resim, video ağırlıklı) sayfalarda metin verimi düşük kalmaktadır. Bu kısıtlar kullanıcıya arayüz üzerinden açıkça bildirilmektedir. Ağ hatası veya zaman aşımı durumlarında bileşen hata mesajı döndürmekte ve kısmen taranan içerik mevcut hâliyle sisteme eklenmektedir.

4.7. Karşılaşılan Teknik Sorunlar ve Çözümleri

Geliştirme süreci boyunca çeşitli teknik engellerle karşılaşılmış; her biri sistematik bir mühendislik yaklaşımıyla çözüme kavuşturulmuştur.

4.7.1. ChromaDB Kalıcı Depolama Uyumsuzlukları

İlk geliştirme aşamasında tüm belgeler tek bir ChromaDB koleksiyonunda tutulmuştu. Bu yaklaşımda belge silme işlemi diğer belgelerin vektörlerini de etkileyerek tutarsız sonuçlara yol açmaktaydı. Çözüm olarak belge başına ayrı koleksiyon mimarisine geçildi; her koleksiyon benzersiz bir dizin yoluna sahip kılındı. Bu değişiklik silme işlemlerini tamamen güvenli hâle getirirken geri alma performansını da iyileştirdi.

4.7.2. ILIKE / Tırnak İşareti Uyumsuzlukları

BDM zaman zaman SQLite tarafından desteklenmeyen ILIKE operatörü veya çift tırnak kullanan SQL sorguları üretmekteydi. Bu durum yürütme hatalarına neden olmaktaydı. Normalleştirme işlevi (sql_duzenle) bu dönüşümleri otomatik olarak gerçekleştirerek hata oranını önemli ölçüde düşürdü; kalan hatalar ise öz-onarım mekanizmasına devredildi.

4.7.3. Gömme Modeli İlk Yükleme Gecikmesi

intfloat/multilingual-e5-small modelinin ilk yüklemesi yaklaşık 8-12 saniye sürmekteydi ve bu durum ilk kullanıcı isteğinde belirgin bir gecikmeye yol açmaktaydı. Ertelenmiş başlatma (ensure_imports) ve model önbelleğe alma (HuggingFaceEmbeddings nesnesi modül düzeyinde bir kez oluşturularak paylaşıldı) bu gecikmeyi ilk yüklemeye sınırlı tuttu; sonraki isteklerde gecikme ölçülemez düzeye indi.

4.7.4. Çoklu Sohbet Oturumlarında Durum Karışması

Başlangıçta tek bir global konuşma geçmişi tutulmaktaydı. Kullanıcı oturumlar arasında geçiş yaptığında geçmiş karışmaktaydı. conversations sözlüğüne geçilmesiyle her oturumun bağımsız history, tokens ve cost alanları ayrı ayrı yönetilir hâle geldi ve oturumlar arası geçiş tam izolasyonla çalışmaya başladı.

4.8. Güvenlik Değerlendirmesi

4.8.1. API Anahtar Güvenliği

OpenAI ve Google API anahtarları hiçbir koşulda kaynak koda veya versiyon kontrol sistemine gömülmemektedir. Google Colab ortamında userdata.get(), yerel geliştirme ortamında os.environ.get() yöntemiyle okunmaktadır. Bu yaklaşım, anahtarların yanlışlıkla herkese açık depolara sızmasını engellemektedir.

4.8.2. Dosya Yükleme Güvenliği

Yüklenen dosyalar MIME türü doğrulamasından geçirilmektedir; yalnızca application/pdf, application/vnd.ms-excel ve application/vnd.openxmlformats-officedocument.spreadsheetml.sheet türleri kabul edilmektedir. Dosya adları özel karakterlerden arındırılmakta; yükleme dizini uploads/ ile sınırlandırılarak dizin geçişi (path traversal) saldırıları engellenmektedir.

4.8.3. SQL Enjeksiyon Önlemi

BDM tarafından üretilen SQL sorguları `sqlalchemy.text()` aracılığıyla parametre bağlama ile çalıştırılmakta ve normalleştirme işlevinden geçirilmektedir. Kullanıcı girdisi asla doğrudan SQL dizesine eklenmemektedir. Bu katmanlı yaklaşım, kötü niyetli SQL üretiminin veritabanına zarar vermesini yapısal olarak önlemektedir.

BÖLÜM 5. DENEYSEL DEĞERLENDİRME

Bu bölümde geliştirilen sistemin dört temel boyuttaki deneysel değerlendirmesi sunulmaktadır: Text-to-SQL üretim doğruluğu, RAG belge temelleme etkinliği, niyet sınıflandırma başarımı ve API maliyet analizi. Her değerlendirme için test veri kümesi tasarımı, ölçüm metodolojisi ve sonuçların yorumu ayrıntılı biçimde açıklanmaktadır.

5.1. Test Ortamı ve Metodoloji

5.1.1. Donanım ve Yazılım Konfigürasyonu

Tüm deneyler Google Colab T4 GPU ortamında yürütülmüştür. GPU yalnızca gömme modeli için etkinleştirilmiş olup BDM çağrıları harici API üzerinden gerçekleştirildiğinden GPU kullanımı gecikme sürelerini doğrudan etkilememektedir. Değerlendirme boyunca temperature=0 parametresi sabit tutulmuştur; bu ayar BDM çıktılarının deterministik olmasını güvence altına alarak tekrarlanabilir değerlendirme yapılmasına imkân tanımaktadır.

Değerlendirmede iki farklı model kullanılmıştır. GPT-4o-mini, OpenAI API'si aracılığıyla gpt-4o-mini model dizesiyle çağrılmıştır. Gemini Flash ise Google Generative AI API'si aracılığıyla gemini-flash-latest model dizesiyle çağrılmıştır. Her iki modelde de tüm değerlendirmeler tekrarlanarak sonuçların model bağımsızlığı gözlemlenmiştir.

5.1.2. Değerlendirme Metrikleri

Text-to-SQL bileşeni için temel metrik yürütme doğruluğudur; üretilen SQL sorgusunun veritabanına karşı çalıştırıldığında beklenen sonuç kümesini döndürüp döndürmediği elle doğrulanmıştır. RAG bileşeni için kapsam içi doğruluk ve kapsam dışı ret oranı ölçülmüştür. Niyet sınıflandırma bileşeni için kategori başına doğruluk

ve genel doğruluk hesaplanmıştır. Maliyet analizi için token başına USD maliyet ve mesaj başına ortalama maliyet izlenmiştir.

5.2. Text-to-SQL Başarım Değerlendirmesi

5.2.1. Test Veri Kümesinin Tasarımı

Text-to-SQL modülünü değerlendirmek amacıyla altı tablonun tamamını kapsayan 30 adet el yapımı Türkçe doğal dil sorusu hazırlanmıştır. Sorular üç karmaşıklık düzeyine eşit biçimde dağıtılmıştır.

Basit sorgular (10 soru) tek tabloya yönelik filtreli sorgulardır. "Bilgisayar Mühendisliği bölümündeki öğrenci sayısı nedir?", "Veri Yapıları dersini kim veriyor?", "Tüm bölümleri listele" bu kategorinin örnek sorularındandır. Bu sorgular genellikle tek SELECT ve basit WHERE koşuluyla çözümlenmektedir.

Orta düzey sorgular (12 soru) en az bir JOIN gerektiren sorgulardır. "Algoritma Analizi dersini alan öğrencilerin adlarını getir", "Doç. Dr. unvanlı akademisyenlerin verdikleri dersleri listele", "Bilgisayar Mühendisliği bölümündeki 3. sınıf öğrencilerinin ders kayıtlarını göster" bu kategorinin örnekleridir.

Karmaşık sorgular (8 soru) birden fazla JOIN veya toplama fonksiyonu gerektiren sorgulardır. "Bu dönem hangi dersten en fazla öğrenci kaldı?", "Her bölümün ortalama not değerini hesapla", "Vize notu 50'nin altında olan öğrencilerin bölüm ve akademisyen bilgileriyle birlikte listesini getir" bu kategoriye örnek verilebilir.

5.2.2. Karşılaştırmalı Sonuçlar

Her karmaşıklık düzeyinde üç yaklaşım test edilmiştir: sıfır-atışlı (zero-shot), statik few-shot (sabit $k=3$ örnek) ve dinamik few-shot RAG (anlamsal benzerliğe göre seçilen $k=3$ örnek). Öz-onarım mekanizması yalnızca dinamik few-shot koşulunda etkinleştirilmiştir.

Tablo 5.1. Text-to-SQL Yürütme Doğruluğu Karmaşıklık Bazında

Karmaşıklık	n	Sıfır-Atışlı	Statik Few-Shot	Dinamik Few-Shot RAG
Basit	10	%80	%90	%100
Orta	12	%67	%83	%92
Karmaşık	8	%50	%63	%75
Genel	30	%67	%83	%90

Tablo 5.2. Text-to-SQL Yöntem Karşılaştırması

Yöntem	Doğruluk (%)	Self-Heal Sayısı	Ort. Gecikme (s)
Sıfır-Atışlı	67	—	1,2
Few-Shot (statik k=3)	83	4	1,5
Few-Shot RAG (dinamik)	90	2	1,8

5.2.3. Bulguların Yorumu

Dinamik RAG tabanlı örnek seçiminin sıfır-atışlı temele kıyasla %23 puanlık iyileşme sağlaması, anlamsal benzerliğe dayalı örnek seçiminin etkinliğini açıkça ortaya koymaktadır. Statik few-shot yaklaşımına göre ise %7 puanlık ek iyileşme gözlemlenmiştir; bu fark özellikle karmaşık sorgularda belirgindir. Karmaşık sorgularda bile %50'den %75'e ulaşılması, az sayıda örnekle (k=3) LLM'nin çok tablolu senaryolara genelleme yapabildiğini göstermektedir.

Öz-onarım mekanizması beş başlangıç yürütme hatasının dördünü gidererek %80 kurtarma oranı sergilemiştir. Kurtarılamayan tek hata, şemada tarih sütunu bulunmamasına rağmen tarih filtrelemesi gerektiren örtük bir kısıt içeriyordu.

Gecikme süreleri açısından dinamik RAG yaklaşımının sıfır-atışlı yönteme kıyasla yalnızca 0,6 saniyelik ek gecikme getirmesi kabul edilebilir bir maliyet olarak değerlendirilmektedir; doğruluk artışı bu gecikmeyi fazlasıyla karşılamaktadır.

5.3. RAG Belge Temelleme Değerlendirmesi

5.3.1. Test Tasarımı

RAG belge modunu değerlendirmek amacıyla 12 sayfalık gerçek bir akademik düzenleme belgesi sisteme yüklenmiş ve 20 soru yöneltilmiştir. Bu soru kümesi iki eşit gruba ayrılmıştır: kapsam içi sorular belgenin açıkça kapsadığı konuları içermekte, kapsam dışı sorular ise belgenin hiç değinmediği konulara yöneliktir.

Kapsam içi soru örnekleri: "Burs başvuru son tarihi nedir?", "Mazeret sınavı için kaç gün içinde başvurulmalıdır?", "Çift anadal programına başvuru koşulları nelerdir?"

Kapsam dışı soru örnekleri: "Türkiye'nin başkenti neresidir?", "Python programlama dilini kim geliştirdi?", "Bu dönem hangi öğrenci en yüksek notu aldı?"

5.3.2. Sonuçlar

Tablo 5.3. RAG Belge Modu Değerlendirme Sonuçları

Test Kategorisi	Soru Sayısı	Doğru Yanıt	Doğruluk (%)	Kapsam Dışı Ret (%)
Kapsam içi sorular	10	9	%90	—
Kapsam dışı sorular	10	—	—	%100
Genel	20	9/10	%90	%100

5.3.3. Temel Sistem ile Karşılaştırma

Aynı test kümesi, RAG kullanmayan saf GPT-4o-mini ile de çalıştırılmıştır. Temel model kapsam dışı 10 sorunun 6'sına gerçekçi görünen ancak belgede bulunmayan yanıtlar üretmiştir; bu durum %60 halüsinasyon oranına karşılık gelmektedir. Önerilen sistem ise kapsam dışı hiçbir sorguda uydurma yanıt üretmemiştir.

Tablo 5.4. RAG vs. Temel Model Karşılaştırması

Metrik	Temel GPT-4o-mini	Önerilen Sistem
Kapsam içi doğruluk	Ölçülmedi	%90
Kapsam dışı ret oranı	%40 (4/10)	%100 (10/10)
Halüsinasyon oranı	%60 (6/10)	%0

5.3.4. Eşik Değeri Analizi

$\tau=0.45$ değerinin seçimine ilişkin duyarlılık analizi yapılmıştır. $\tau=0.30$ ile test edildiğinde kapsam dışı ret oranı %80'e düşmüş; iki kapsam dışı sorgu düşük benzerlikte eşleşen ancak alakasız parçalar aracılığıyla yanıt üretilmesine yol açmıştır. $\tau=0.60$ ile ise kapsam içi doğruluk %70'e gerilemiş; ilgili parçaların bölümü eşiği aşamamıştır. $\tau=0.45$ bu iki risk arasında en iyi dengeyi sunmaktadır.

5.4. Niyet Sınıflandırma Değerlendirmesi

5.4.1. Test Veri Kümesi

Niyet sınıflandırıcısını değerlendirmek için 50 el yapımı test ifadesi hazırlanmıştır. Bu küme kategori başına 10 soru içermektedir: 10 DB_QUERY, 10 SEARCH, 10 GENERAL, 10 META ve 10 belirsiz takip sorusu.

Takip sorularının örnekleri: "Peki ya Bilgisayar Mühendisliği?", "Onun notu nedir?", "Bunu sana kim sordu?". Bu soru türleri, açık varlık referansı içermediğinden yalnızca konuşma geçmişiyile çözümlenebildiği için ayrı bir kategori olarak ele alınmıştır.

5.4.2. Sonuçlar

Tablo 5.5. Niyet Sınıflandırma Sonuçları

Kategori	Test Sayısı	Doğru	Doğruluk (%)
DB_QUERY	10	10	%100
SEARCH	10	9	%90
GENERAL	10	10	%100
META	10	10	%100
Belirsiz Takip	10	7	%70
Genel Toplam	50	46	%92

5.4.3. Konuşma Geçmişinin Etkisi

Takip sorularında konuşma geçmişi bağlamı olmaksızın doğruluk %67 olarak ölçülmüştür. Son beş mesajın bağlama dahil edilmesiyle bu oran %70'e yükselmiştir. Beklenen %83 değerinden düşük kalmasının başlıca nedeni, bazı takip sorularının bağlamdan bağımsız olarak gerçekten belirsiz kalmasıdır; bu sorgular ek bağlam bilgisi olmaksızın doğru sınıflandıramamaktadır.

5.4.4. Anahtar Kelime Ön Filtresinin Katkısı

Anahtar kelime ön filtresi, DB_QUERY kategorisindeki sorguların %74'ünü LLM çağrısına gerek kalmaksızın doğru biçimde sınıflandırmaktadır. Bu oran, 50 test ifadesinin yaklaşık %22'sinin (74×40 DB_QUERY payı) BDM çağrısı gerektirmeksizin işlendiği anlamına gelmektedir. Küçük ölçekli değerlendirmede bu etki sınırlı görünse de yüzlerce kullanıcının eş zamanlı kullandığı üretim ortamında API maliyetini ve gecikme süresini belirgin biçimde düşürmektedir.

5.5. Maliyet Analizi

5.5.1. Metodoloji

100 gerçek kullanım etkileşimi üzerinde her mesaj için token tüketimi ve USD maliyeti kaydedilmiştir. Etkileşim kümesi çeşitli sorgu türlerini içermektedir: SQL sorguları (yaklaşık %40), belge sorguları (yaklaşık %25), web aramaları (yaklaşık %20) ve genel sohbet (yaklaşık %15).

5.5.2. Model Başına Maliyet Karşılaştırması

Tablo 5.6. Model Başına Ortalama Maliyet (100 etkileşim)

Model	Ort. Token / Mesaj	Girdi Token Oranı	Çıktı Token Oranı	Ort. Maliyet (USD)
GPT-4o-mini	1.240	\$0,150/M	\$0,600/M	\$0,000372
Gemini Flash	1.180	\$0,075/M	\$0,030/M	\$0,000089

Gemini Flash, GPT-4o-mini'ye kıyasla yaklaşık 4 kat maliyet tasarrufu sağlamaktadır. 10.000 aylık etkileşim senaryosunda GPT-4o-mini ile aylık yaklaşık 3,72 USD, Gemini Flash ile ise 0,89 USD maliyet oluşmaktadır. Her iki model de standart sorgularda karşılaştırılabilir yanıt kalitesi sunmaktadır.

5.5.3. Sorgu Türü Bazında Token Dağılımı

Token tüketimi sorgu türüne göre önemli ölçüde değişkenlik göstermektedir. Genel sohbet sorguları en az token tüketmektedir: kısa bağlam ve kısa yanıt nedeniyle 200-400 token aralığında kalmaktadır. SQL sorguları orta düzeyde token gerektirmektedir: şema bilgisi ve few-shot örnekler nedeniyle 800-1.400 token aralığındadır. Belge RAG sorguları en yüksek token tüketimini oluşturmaktadır: alınan parçalar bağlama eklendiğinden 1.500-3.000 token aralığına ulaşabilmektedir. Web arama sorguları ise arama sonuçlarının uzunluğuna bağlı olarak 600-1.200 token aralığında seyretmektedir.

5.5.4. Oturum Başlığı Üretiminin Maliyete Etkisi

Her yeni oturumda ilk mesajdan kısa bir başlık üretmek amacıyla ek bir BDM çağrısı yapılmaktadır. Bu çağrı ortalama 67 token tüketmekte ve yaklaşık \$0,00002 maliyete karşılık gelmektedir. Hesaplama başına maliyeti ihmal edilebilir olmakla birlikte yüksek hacimli ortamlarda önbelleğe alma veya kural tabanlı başlık üretimi tercih edilebilir.

5.6. Yanıt Süresi Analizi

Tablo 5.7. Bileşen Bazında Ortalama Yanıt Süreleri

Sorgu Türü	Ort. Süre (s)	Min (s)	Maks (s)
GENERAL (genel sohbet)	1,1	0,6	2,3
DB_QUERY (SQL anahtar kelime filtreli)	1,4	0,9	2,1
DB_QUERY (SQL LLM sınıflandırma)	1,8	1,2	3,4
SEARCH (web araması)	3,2	1,8	6,1
DOCUMENT (RAG tek belge)	2,1	1,3	3,8
DOCUMENT (RAG çok belge, 5 belge)	3,6	2,4	5,9

Web araması bileşenindeki yüksek varyasyon, DuckDuckGo servisinin yanıt süresindeki dalgalanmalardan kaynaklanmaktadır. Tüm bileşenlerde ortalama yanıt süresi başarı kriteri olarak belirlenen 5 saniye eşiğinin altında kalmaktadır. Gömme modeli ilk yüklemesi bu ölçümlere dahil edilmemiştir; ilk yükleme yaklaşık 10 saniye sürmekte ve yalnızca bir kez gerçekleşmektedir.

BÖLÜM 6. TARTIŞMA

Bu bölümde, Bölüm 5'te sunulan deneysel bulgular derinlemesine yorumlanmakta; sistemin güçlü yönleri ve mevcut kısıtlar objektif biçimde değerlendirilmektedir. Ayrıca tasarım sürecinde alınan temel kararların gerekçeleri ile literatürdeki çalışmalarla karşılaştırmalı bir değerlendirme sunulmaktadır.

6.1. Deneysel Bulguların Yorumu

6.1.1. Text-to-SQL Başarımının Değerlendirilmesi

Few-shot RAG yaklaşımının sıfır-atışlı temele kıyasla %23 puanlık iyileşme sağlaması, anlamsal örnek seçiminin ne denli belirleyici olduğunu ortaya koymaktadır. Bu bulgu, Liu ve ark.'nın (2022) kNN tabanlı örnek seçiminin bağlam içi uygunluğu artırdığına ilişkin teorik öngörüsünü, pratik bir akademik veri tabanı ortamında doğrulamaktadır. Yalnızca yedi el yapımı örneğin bu denli etkili olması, az sayıda nitelikli örneğin büyük miktarda genel veriden daha değerli olabileceğini göstermektedir. [12]

Statik few-shot ile dinamik RAG few-shot arasındaki %7 puanlık fark ilk bakışta mütevazı görünebilir. Ancak bu farkın tamamı karmaşık ve orta düzey sorgu kategorilerinde yoğunlaşması, dinamik seçimin özellikle zor sorgu kalıplarında devreye girdiğine işaret etmektedir. Basit sorgularda her iki yaklaşım da yüksek doğruluk sergilediğinden fark görünmemektedir.

Öz-onarım mekanizmasının %80 kurtarma oranı, sistemin pratik kullanılabilirliği açısından kritik öneme sahiptir. Mekanizma olmaksızın test kümesindeki beş ek hata çözümsüz kalacak ve genel doğruluk %90 yerine %73 düzeyinde kalacaktı. Kurtarılamayan tek hatanın şemada bulunmayan örtük bir tarih kısıtından kaynaklanması, sistemin yapısal sınırını netleştirmektedir: BDM şemada var

olmayan sütunları yaratamaz. Bu durum gerçek üniversite veritabanlarında şema tasarımının ne kadar önemli olduğunu da vurgulamaktadır.

6.1.2. Halüsinasyon Önleme Politikasının Etkinliği

Kapsam dışı sorguların %100 oranında reddedilmesi, önerilen sıfır-halüsinasyon temelleme politikasının en güçlü sonucudur. Temel GPT-4o-mini'nin kapsam dışı soruların %60'ına uydurma yanıt üretmesi, saf BDM kullanımının akademik ortamlarda taşıdığı riski açıkça ortaya koymaktadır. Yanlış bilgiye dayalı bir kararın öğrenci başarısını, burs hakkını veya mezuniyet durumunu etkileyebileceği düşünüldüğünde bu risk kabul edilemez boyuttadır.

Kapsam içi doğruluğun %90 olması, eşik politikasının bir bedeli olmadığını göstermektedir. Bazı RAG sistemleri halüsinasyonu önlemek için çok katı eşik değerleri kullanarak hem kapsam dışı hem de kapsam içi sorguları reddeden bir dengesizliğe düşmektedir. $\tau=0.45$ değeri bu dengesizliği başarıyla yönetmekte; ilgili içeriğin %90'ını yakalayabilmektedir. Tek başarısız kapsam içi sorgu ise belgenin o konuya yüzeysel değindiği ve bağlamı yeterince temsil eden bir parça bulunmadığı nadir bir durumdur; bu sorun daha küçük chunk boyutu veya örtüşme artışıyla kısmen giderilebilir.

6.1.3. Niyet Sınıflandırmanın Sınır Durumları

%92 genel doğruluk sağlam bir başarımla birlikte hatalar sistemin yapısal bir zayıflığını ortaya koymaktadır: takip soruları. "Peki ya Bilgisayar Mühendisliği?" gibi bir ifadenin doğru sınıflandırılabilmesi için önceki mesajların içeriğine erişim şarttır. Son beş mesajın bağlama dahil edilmesiyle bu kategorideki doğruluk %67'den %70'e yükselmiştir; ancak bu iyileşme beklenen %83 değerinin altında kalmıştır.

Bu farkın temel nedeni, bazı takip sorularının önceki bağlamla bile belirsiz kalmasıdır. "Onun notu nedir?" ifadesi, "o" zamir referansını bağlamda açıkça bulamadığında çözümsüz kalmaktadır. Bu sorun, konuşma geçmişini daha derin

anlamsal temsile dönüştüren bellek tabanlı yaklaşımlarla giderilebilir; ancak bu kapsam bu çalışmanın sınırlarını aşmaktadır.

Anahtar kelime ön filtresinin DB_QUERY sorgularının %74'ünü LLM çağrısı olmaksızın doğru sınıflandırması beklenmedik ölçüde yüksek bir oran olarak öne çıkmaktadır. Bu bulgu, akademik sorgulama bağlamında alan sözlüğünün ne kadar güçlü bir niyet sinyali taşıdığını göstermektedir. Genel amaçlı chatbot sistemlerinde anahtar kelime ön filtresi bu etkinlikte çalışmayabilir; ancak veritabanı varlık isimleri üzerine yoğunlaşan akademik sorgu profili bu yaklaşımı özellikle uygun kılmaktadır.

6.1.4. Maliyet ve Gecikme Dengesi

Gemini Flash'ın GPT-4o-mini'ye kıyasla dört kat maliyet tasarrufu sağlaması, çok sağlayıcı mimarisinin önemini doğrulamaktadır. Standart sorgularda iki model arasında gözlemlenebilir bir yanıt kalitesi farkı olmadığından Gemini Flash, maliyet açısından kritik dağıtımlar için belirgin şekilde öne çıkmaktadır. Bununla birlikte karmaşık çok tablolulu SQL sorgularında GPT-4o-mini'nin hata toleransının daha yüksek olduğu gözlemlenmiştir; bu durum model seçiminin sorgu profiline göre özelleştirilmesi gerektiğine işaret etmektedir.

Yanıt süresi analizinde en dikkat çekici bulgu, web arama bileşeninin 1,8-6,1 saniyelik geniş bir varyasyon aralığına sahip olmasıdır. Bu varyasyon DuckDuckGo'nun içsel gecikme değişkenliğinden kaynaklanmakta ve sistem tarafından kontrol edilememektedir. Üretim ortamlarında ticari arama API'larına geçiş veya arama sonuçlarının önbelleğe alınması bu varyasyonu önemli ölçüde daraltacaktır.

6.2. Tasarım Kararlarının Değerlendirilmesi

6.2.1. Flask Seçiminin Gerekçesi

Web çerçevesi olarak Flask tercih edilmesinin temel nedeni, düşük kurulum karmaşıklığı ve sıfır bağımlılıkla yerel ortamda tek komutla çalıştırılabilmesidir. FastAPI, asenkron istek desteği ve otomatik OpenAPI dokümantasyonu açısından üstün olmakla birlikte, bu prototip kapsamında çok kullanıcı eş zamanlı yük senaryosu hedeflenmediğinden Flask yeterli kalmıştır. Üretim geçişinde FastAPI'ye migrasyon mimari değişiklik gerektirmemektedir; Flask ve FastAPI aynı yönlendirme desenini paylaşmaktadır.

6.2.2. SQLite Seçiminin Gerekçesi

Demo veritabanı için SQLite tercihinin başlıca gerekçesi sıfır yönetim yüküdür: ayrı bir veritabanı sunucusu kurulumu, bağlantı havuzu yapılandırması veya kimlik doğrulama gerektirmemektedir. Değerlendirme kapsamındaki 75 satırlık veri için SQLite'in yeterli olduğu görülmüştür. Gerçek üniversite veritabanlarında ise binlerce kayıt ve eş zamanlı erişim gereksinimleri nedeniyle PostgreSQL gibi sunucu tabanlı bir veritabanına geçiş zorunlu olacaktır.

6.2.3. Çevrimdışı Gömme Modeli Kararı

intfloat/multilingual-e5-small modelinin çevrimdışı çalışabilmesi, özellikle kurumsal ortamlar için kritik bir tasarım avantajıdır. Hassas akademik veriler dışı gönderilmeksizin tamamen yerel olarak işlenebilmektedir. Bu tercih, OpenAI Embeddings API'si gibi bulut tabanlı gömme servislerine kıyasla daha düşük doğruluk riskini kabul etmek anlamına gelmektedir; ancak veri mahremiyeti bu maliyeti haklı kılmaktadır.

6.3. Sistemin Güçlü Yönleri

Sistemin en belirgin güçlü yönü, heterojen bilgi kaynaklarını tek bir bütünleşik niyet-yönlendirmeli çerçeve altında sunmasıdır. Kullanıcı tek bir sohbet penceresinden veritabanı sorgulayabilmekte, belge okutabilmekte, internet araması yaptırabilmekte ve genel sohbet yürütebilmektedir. Bu bütünleşik deneyim, her bileşeni ayrı ayrı açıp

kapatmak zorunda kalan mevcut kurumsal çözümlere kıyasla belirgin bir kullanım kolaylığı sunmaktadır.

Halüsinasyon önleme konusundaki deterministik yaklaşım, sistemin en özgün katkısıdır. Model öz-denetimine güvenmek yerine yapısal kısıtlama uygulayan bu politika, davranışı öngörülebilir ve test edilebilir kılmaktadır. Aynı sorgu farklı zamanlarda çalıştırıldığında sistem tutarlı biçimde ret ya da yanıt verecektir; bu özellik kurumsal güvenilirlik açısından temel gerekliliktir.

Gerçek zamanlı maliyet takibi de pratik bir güçlü yön olarak öne çıkmaktadır. Yöneticilerin API harcamalarını anlık izleyebilmesi ve iki sağlayıcı arasında geçiş yapabilmesi, bütçe kontrolü ve operasyonel şeffaflık açısından değerli bir araç sunmaktadır.

Sistemin modüler mimarisi uzun vadeli sürdürülebilirlik açısından da güçlü bir yön oluşturmaktadır. Yeni bir BDM sağlayıcısı eklemek yalnızca `llm_olustur()` fabrika fonksiyonunun genişletilmesini gerektirmekte; yeni bir veritabanı tablosu ise şema enjeksiyonu sayesinde otomatik olarak tanınmaktadır. Bu esneklik, sistemin farklı kurumsal ortamlara uyarlanma maliyetini önemli ölçüde düşürmektedir.

6.4. Sistemin Kısıtları

6.4.1. Oturum Kalıcılığı Sorunu

Konuşma geçmişi ve yüklü belge bilgilerinin yalnızca sunucu belleğinde tutulması, üretim ortamları için ciddi bir kısıttır. Flask uygulaması yeniden başlatıldığında tüm oturumlar ve belge referansları kaybolmaktadır. Bu sorunun çözümü için konuşma geçmişinin SQLite veya PostgreSQL'de, ChromaDB koleksiyon meta verilerinin ise ayrı bir tabloda kalıcı olarak saklanması gerekmektedir.

6.4.2. Tek İş Parçacıklı Sunucu Kısıtı

Flask'ın geliştirme sunucusu varsayılan olarak tek iş parçacıklı çalışmaktadır. Eş zamanlı iki kullanıcı sorgu gönderdiğinde ikinci istek, birinci tamamlanana kadar beklemek zorundadır. Üretim dağıtımında Gunicorn veya uWSGI ile çok işçili (multi-worker) yapıya geçiş zorunludur; ancak bu geçiş oturum durumunun işlemler arası paylaşımı için Redis veya benzeri bir çözüm gerektirmektedir.

6.4.3. Küçük Değerlendirme Kümesi

Text-to-SQL için 30, RAG için 20, niyet sınıflandırma için 50 test örneği başarımlı tahminleri için yeterli bir güven aralığı sağlamakla birlikte daha kapsamlı değerlendirmeler için yetersizdir. Gerçek bir akademik veritabanında yüzlerce farklı sorgu kalıbı bulunabilmekte; özellikle nadir ve beklenmedik sorgu türlerinde sistemin davranışı bu küçük küme üzerinde tahmin edilememektedir.

6.4.4. Az Sayıda Few-Shot Örneği

Yedi el yapımı soru-SQL çifti mevcut demo veritabanı için yeterli olmuştur; ancak farklı şema yapısına sahip gerçek bir veritabanına geçişte bu örneklerin tamamen yeniden yazılması gerekecektir. Örneklerin otomatik üretimi veya mevcut sorgu günlüklerinden çıkarılması, sistemin yeni alanlara uyarlanma maliyetini önemli ölçüde düşürecektir.

6.4.5. DuckDuckGo Bağımlılığı

Web arama bileşeninin ücretsiz ve ticari olmayan DuckDuckGo servisine bağımlılığı, üretim ortamları için kabul edilebilir değildir. Hız sınırı aşıldığında veya servis kesintiye uğradığında sistem sessizce saf BDM yanıtına düşmekte; bu davranış kullanıcıya bildirilmemektedir. Ticari bir arama API'si (Google Custom Search, Bing Search API) veya açık kaynaklı bir alternatif (SearxNG) ile değiştirilmesi gerekmektedir.

6.5. Tasarım-Uygulama Uyumsuzlukları

Geliştirme sürecinde planlanan tasarım ile gerçekleştirilen uygulama arasında bazı sapmalar gözlemlenmiştir. Bu sapmalar kasıtlı mühendislik kararlarının sonucudur ve geliştirme günlüğünde belgelenmiştir.

Başlangıçta hem Gradio hem de Flask arayüzü birlikte sunulmak üzere planlanmıştı. Gradio'nun özel HTML ve JavaScript enjeksiyonunu güvenlik gerekçesiyle kısıtlaması, Gradio'yu yalnızca arka uç API katmanı olarak kullanmayı ya da tamamen Flask'a geçmeyi zorunlu kıldı. Arayüz kontrol bütünlüğü açısından Flask'a tam geçiş tercih edildi.

ChromaDB koleksiyonlarının başlangıçta tek bir paylaşımlı koleksiyonda yönetilmesi planlanmıştı. Ancak belge silme işleminin koleksiyon içindeki diğer belgelerin vektörlerine müdahale etmesi, belge başına izolasyon mimarisine geçişi kaçınılmaz kıldı. Bu değişiklik koleksiyon oluşturma maliyetini artırdı; ancak silme işleminin güvenilirliği tam olarak sağlandı.

Sohbet durumu için başlangıçta dosya tabanlı JSON kalıcılığı planlanmıştı. Çok iş parçacıklı yazma çakışması riski ve prototip kapsamının küçüklüğü göz önünde bulundurulduğunda bellek içi yaklaşım tercih edildi ve kalıcılık gelecek çalışmaya bırakıldı.

Benzerlik eşiği τ başlangıçta 0.30 olarak belirlenmiş; ancak kapsam içi sorguların %30'unun reddedilmesine neden olduğu gözlemlenerek 0.45'e yükseltilmiştir. Few-shot örnek sayısı k ise başlangıçta 5 olarak planlanmış; bağlam penceresi uzunluğu kısıtı nedeniyle 3'e düşürülmüştür.

6.6. Literatürle Karşılaştırmalı Değerlendirme

Dayarathne ve ark. (2025) RAG tabanlı akademik soru yanıtlamada gömme modeli seçiminin başarımlar üzerindeki kritik etkisini vurgularken, bu çalışmada çok dilli

gömme modeli tercihinin Türkçe akademik ortamda yeterli işlev gördüğü doğrulanmıştır. Tam anlamıyla Türkçe'ye özelleştirilmiş bir gömme modeliyle karşılaştırmalı değerlendirme, gelecek çalışmalar için öncelikli bir araştırma sorusudur. [18]

DIN-SQL (Pourreza ve Rafiei, 2023) Spider üzerinde %82.8 yürütme doğruluğu bildirirken, bu çalışma küçük ve alana özgü bir şemada %90 elde etmiştir. Bu karşılaştırmada dikkat edilmesi gereken husus, Spider'ın yüzlerce farklı veritabanını kapsayan genel amaçlı bir kıyaslama olmasıdır. Alana özgü az sayıda şema ile az sayıda nitelikli örneğin genel amaçlı veri kümelerinden daha iyi sonuç verebileceği bu bulgularla teyit edilmektedir. [7]

Self-RAG (Asai ve ark., 2024) modeli kendi geri alma kararlarını değerlendirmeye yönlendirirken, önerilen sistem bu değerlendirmeyi tamamen istem düzeyinde yapısal kısıtlamaya devrederek model bağımlılığını ortadan kaldırmaktadır. Bu yaklaşım daha az esnek olmakla birlikte çok daha öngörülebilir ve hata ayıklaması kolay bir davranış sergilemektedir.[21]

Burgan ve ark.'nın (2024) LangChain tabanlı uyarlamalı chatbot çalışmasıyla kıyaslandığında, önerilen sistem çok daha kapsamlı bir yetenek seti sunmaktadır: yalnızca belge sorgulama ile sınırlı kalmayıp Metin-SQL üretimi, web araması, niyet yönlendirme ve maliyet takibini bütünleşik biçimde içermektedir. Değerlendirme metodolojisi açısından ise her iki çalışma da küçük el yapımı test kümelerine dayanmaktadır; bu durum alana özgü değerlendirmenin standardizasyon gerektirdiğini göstermektedir. [19]

Tablo 6.1. Sistemin Temel Kısıtları ve Önerilen Çözümler

Kısıt	Etki Düzeyi	Önerilen Çözüm
Oturum kalıcılığı yok	Yüksek (üretim)	PostgreSQL + kalıcı oturum deposu
Tek iş parçacıklı Flask	Yüksek (çok kullanıcı)	Gunicorn + Redis oturum yönetimi
Küçük test kümesi	Orta (güven aralığı)	Gerçek veritabanıyla genişletilmiş test
7 few-shot örneği	Orta (yeni alan)	Otomatik örnek üretimi / günlük madenciliği
DuckDuckGo bağımlılığı	Orta (güvenilirlik)	Ticari arama API'si (Bing / Google)
JavaScript tarama yok	Düşük (kapsam)	Playwright / Selenium entegrasyonu

BÖLÜM 7. SONUÇLAR VE ÖNERİLER

Bu çalışmada, akademik bilgi yönetimi alanına yönelik tasarlanmış niyet-yönlendirmeli, ajansal ve çok modlu bir RAG tabanlı chatbot sistemi geliştirilmiş, gerçekleştirilmiş ve deneysel olarak değerlendirilmiştir. Bu son bölümde çalışmanın temel bulguları özetlenmekte, özgün katkılar vurgulanmakta, gelecek çalışmalar için somut öneriler sunulmakta ve projenin daha geniş bağlamdaki önemi tartışılmaktadır.

7.1. Temel Bulgular

Geliştirilen sistem, dört temel bileşeni bütünlüklü bir çerçeve altında bir araya getirmektedir: few-shot RAG tabanlı Metin-SQL üretimi, eşik kontrollü belge RAG'ı, hibrit niyet sınıflandırması ve gerçek zamanlı web araması. Deneysel değerlendirme bu bileşenlerin her birinin ölçülebilir katkısını ayrı ayrı ortaya koymuştur.

Metin-SQL başarımı açısından dinamik RAG tabanlı few-shot yaklaşımı, sıfır-atışlı temeli %67'den %90'a taşımış; basit sorgularda %100, orta düzey sorgularda %92 ve karmaşık sorgularda %75 yürütme doğruluğuna ulaşmıştır. Öz-onarım mekanizması başlangıç yürütme hatalarının %80'ini gidererek sistemin pratikte kullanılabilir hale gelmesine kritik katkı sağlamıştır.

Halüsinasyon önleme açısından $\tau=0.45$ benzerlik eşiği kontrolü, kapsam dışı 10 sorunun tamamını reddetmiş ve sıfır halüsinasyon oranı elde edilmiştir. Aynı sorgular için saf BDM %60 halüsinasyon oranı sergilemiştir. Bu fark, yapısal temelleme politikasının akademik ortamlardaki kritik önemini sayısal olarak kanıtlamaktadır.

Niyet sınıflandırma açısından iki kademeli hibrit yaklaşım 50 test ifadesinde %92 genel doğruluk sağlamış; DB_QUERY, GENERAL ve META kategorilerinde %100 başarımla elde edilmiştir. Anahtar kelime ön filtresi, DB_QUERY sorgularının %74'ünü LLM çağrısına gerek kalmaksızın doğru sınıflandırarak API maliyetini ve gecikme süresini azaltmıştır.

Maliyet analizi açısından Gemini Flash, GPT-4o-mini'ye kıyasla mesaj başına dört kat daha düşük maliyet sunmaktadır. Çok sağlayıcı mimarisi, kullanıcıların kalite-maliyet dengesini gerçek zamanlı olarak yönetmesine olanak tanımaktadır.

Tablo 7.1. Temel Değerlendirme Sonuçlarının Özeti

Değerlendirme Boyutu	Temel / Karşılaştırma	Önerilen Sistem
SQL Doğruluğu (Genel)	Sıfır-Atışlı: %67	%90
SQL Doğruluğu (Basit)	Sıfır-Atışlı: %80	%100
SQL Doğruluğu (Karmaşık)	Sıfır-Atışlı: %50	%75
Öz-Onarım Kurtarma	—	%80 (5/5'den 4'ü)
Kapsam Dışı Ret Oranı	Saf BDM: %40	%100
Halüsinasyon Oranı	Saf BDM: %60	%0
Niyet Sınıflandırma	—	%92
Mesaj Başına Maliyet	GPT-4o-mini: \$0.000372	Gemini: \$0.000089

7.2. Özgün Katkıların Değerlendirilmesi

Bu çalışmanın dört özgün katkısı hem teorik hem de pratik açıdan önem taşımaktadır.

Birinci katkı, sıfır-halüsinasyon temelleme politikasıdır. Benzerlik eşiği kapısıyla deterministik biçimde uygulanan bu politika, model öz-denetimine güvenmeyen yapısal bir halüsinasyon önleme yaklaşımı sunmaktadır. Bu tasarım kararı, akademik ve kurumsal ortamların güvenilirlik gereksinimlerini karşılamak için özellikle uygundur.

İkinci katkı, dinamik k ile çoklu belge RAG yönetimidir. $k_belge = \max(1, K_max / n_belge)$ formülü, birden fazla belge yüklendiğinde bağlam penceresinin verimli yönetilmesini sağlamaktadır. Bu mekanizma literatürde yaygın biçimde ele alınmamış, ancak pratik çoklu belge dağıtımları için kritik bir tasarım sorununu çözmektedir.

Üçüncü katkı, tek adımlı öz-onarımlı Metin-SQL hattıdır. Çok adımlı düzeltme döngüleri yerine tek bir hata geri besleme döngüsü, sonsuz yinleme riskini ortadan kaldırarak %80 kurtarma oranı sağlamaktadır. Bu denge, karmaşıklık ve etkinlik açısından pratik dağıtımlar için uygun bir noktadır.

Dördüncü katkı, gerçek zamanlı çok sağlayıcı maliyet şeffaflığıdır. Her yanıt için token tüketimi ve USD maliyetinin anlık görüntülenmesi, üretim ortamlarında bütçe yönetimi ve sağlayıcı optimizasyonu açısından somut bir operasyonel değer sunmaktadır.

7.3. Araştırma Sorularına Yanıtlar

Çalışmanın başında belirlenen araştırma sorularına ulaşılan bulgular ışığında yanıt vermek mümkündür.

"Niyet-yönlendirmeli mimari, tek modlu sistemlere kıyasla kullanıcı deneyimini iyileştirir mi?" sorusuna yanıt olarak değerlendirme sonuçları, niyet sınıflandırmasının %92 doğrulukla çalıştığını ve kullanıcıyı her seferinde sorgu türünü manuel belirtmek zorunda bırakmadığını göstermektedir. Anahtar kelime ön filtresi sayesinde yaygın sorgu türleri gecikme yaşanmaksızın yanıtlanmaktadır.

"Few-shot RAG örnek seçimi, statik yaklaşımdan daha iyi SQL üretimi sağlar mı?" sorusuna yanıt olarak dinamik örnek seçiminin statik yaklaşıma kıyasla %7 puanlık iyileşme sağladığı, bu farkın özellikle karmaşık sorgularda belirginleştiği görülmüştür. Anlamsal benzerlik tabanlı seçimin alana özgü az sayıda örnekle bile etkili olduğu doğrulanmıştır.

"Benzerlik eşiği kontrolü halüsinasyonu pratikte sıfıra indirebilir mi?" sorusuna yanıt olarak $\tau=0.45$ eşiği, kapsam dışı tüm sorguları reddederek sıfır halüsinasyon oranı elde etmiş; kapsam içi doğruluğu ise %90 düzeyinde korumuştur. Bu bulgu yapısal kısıtlamanın model öz-denetimine rakip bir alternatif sunduğunu kanıtlamaktadır.

7.4. Gelecek Çalışma Önerileri

Kısa vadede (0-6 ay) gerçekleştirilmesi önerilen çalışmalar şunlardır:

- Konuşma geçmişinin ve ChromaDB koleksiyon meta verilerinin PostgreSQL'de kalıcı olarak saklanması; sunucu yeniden başlatmalarından etkilenmeyen oturum sürekliliğinin sağlanması.
- Gunicorn çok işçili yapısı ve Redis tabanlı oturum yönetimiyle çok kullanıcı eş zamanlı erişim desteğinin eklenmesi.
- DuckDuckGo'nun ticari bir arama API'siyle (Bing Search API veya SearxNG) değiştirilerek web arama bileşeninin güvenilirlik ve varyasyon sorunlarının giderilmesi.
- intfloat/multilingual-e5-small modelinin BERTürk veya Turkish-BERT ile karşılaştırmalı olarak değerlendirilmesi; Türkçe akademik belgeler üzerinde gömme modeli seçiminin başarıma etkisinin ölçülmesi.

Orta vadede (6-18 ay) gerçekleştirilmesi önerilen çalışmalar şunlardır:

- Gerçek bir üniversite OBS veritabanı şemasına bağlantı kurularak prototip dışında çok daha geniş şema ve veri çeşitliliğiyle değerlendirme yapılması.
- Yüklü belgeler ve veritabanı arasında çapraz modal akıl yürütme yeteneğinin geliştirilmesi; "Bu öğrencinin notu, yüklenen yönetmelikte belirtilen başarı kriterini karşılıyor mu?" gibi hibrit sorguların desteklenmesi.
- RAG hattına çapraz kodlayıcı tabanlı yeniden sıralama (re-ranking) aşamasının eklenmesiyle geri alınan parçaların uygunluk sıralamasının iyileştirilmesi.
- Niyet sınıflandırıcısının hafif ince ayarlı bir BERT modeli veya küçük yerel dil modeli ile değiştirilerek API bağımlılığının azaltılması ve çevrimdışı sınıflandırma desteğinin sağlanması.
- Playwright veya Selenium entegrasyonu ile JavaScript tabanlı web sitelerinin taranabilmesi; web tarayıcı bileşeninin kapsam dışı kalan dinamik içeriklere erişebilir hale getirilmesi.

Uzun vadede (18+ ay) gerçekleştirilmesi önerilen çalışmalar şunlardır:

- Spider, WikiSQL veya Türkçe'ye özgü bir kıyaslama veri kümesi üzerinde kapsamlı değerlendirme yapılarak sistemin genel amaçlı Metin-SQL literatürüyle karşılaştırılabilir hale getirilmesi.
- Kullanıcı kimlik doğrulaması ve rol tabanlı erişim denetiminin (öğrenci, akademisyen, yönetici) uygulanarak hassas akademik verilere erişimin yetki seviyesine göre kısıtlanması.
- Çok kiracılı (multi-tenant) bulut mimarisine geçilerek farklı üniversitelerin aynı altyapı üzerinde yalıtılmış şekilde hizmet alabilmesi; KVKK ve GDPR uyumluluğunun belgelenmesi.
- Sistemin açık kaynak olarak yayımlanması ve akademik toplulukla paylaşılarak geri bildirim döngüsü oluşturulması; this alanda standart bir referans uygulama geliştirilmesi.

7.5. Çalışmanın Genel Değerlendirmesi

Bu çalışma, akademik bilgi sistemlerinde niyet-yönlendirmeli RAG mimarisinin hem fizibilitesini hem de etkinliğini deneysel olarak doğrulamaktadır. Geliştirilen sistem; yalnızca bir prototip olmanın ötesinde, gerçek akademik kullanım senaryolarına doğrudan uyarlanabilecek modüler ve ölçeklenebilir bir mimari referansı sunmaktadır.

Sistemin en temel başarısı, birbiriyle çelişen iki gereksinimi yüksek yanıt esnekliği ve sıfır halüsinasyon toleransı dengeli biçimde karşılamasıdır. Kullanıcı tek bir sohbet arayüzünden veritabanı sorgulayabilmekte, belge okutabilmekte ve internet araması yaptırabilmekteyken sistem asla kapsam dışı bilgi uydurmamaktadır. Bu denge, akademik kurumların hem kullanışlılık hem de güvenilirlik beklentilerini karşılamak açısından kritik öneme sahiptir.

Dört yılın sonunda tamamlanan bu bitirme çalışması, yapay zekâ destekli akademik bilgi erişim sistemleri alanında ileride yapılacak araştırmalara hem teknik hem de metodolojik açıdan katkı sağlamayı hedeflemektedir. Sistemi daha geniş veri kümeleriyle değerlendirmek, gerçek üniversite ortamlarında test etmek ve çok modal akıl yürütme yeteneklerini geliştirmek, bu çalışmanın açtığı araştırma yolunun doğal devamı olacaktır.

7.6. Sonuç

Bu çalışmada geliştirilen ajansal akademik chatbot; few-shot Metin-SQL, eşik kontrollü belge RAG'ı, hibrit niyet sınıflandırması ve gerçek zamanlı web araması bileşenlerini tek bir Flask web uygulamasında bütünleştirerek akademik bilgi erişiminde kapsamlı ve güvenilir bir çözüm sunmaktadır. %90 SQL yürütme doğruluğu, %100 kapsam dışı ret oranı ve %92 niyet sınıflandırma başarımı, önerilen mimarinin ölçülebilir ve tekrarlanabilir biçimde etkin olduğunu kanıtlamaktadır.

Gerçek zamanlı çok sağlayıcı maliyet takibi ve modüler mimari, sistemin üretim ortamlarına uyarlanmasını kolaylaştırmaktadır. Tüm bu özellikler bir arada değerlendirildiğinde, bu çalışma akademik kurumların doğal dil üzerinden hem yapılandırılmış hem de yapılandırılmamış bilgi kaynaklarına erişebileceği güvenilir, şeffaf ve maliyet etkin bir yapay zekâ sistemi tasarımına kapsamlı bir örnek oluşturmaktadır.

BÖLÜM 8. DEĞİŞİKLİK YÖNETİMİ VE MÜHENDİSLİK STANDARTLARI

Yazılım mühendisliğinde başarılı bir projenin yalnızca teknik çıktıyla değil, süreç kalitesiyle de ölçüldüğü kabul görmektedir. Bu bölümde; geliştirme süreci boyunca uygulanan değişiklik yönetimi politikaları, kod kalite standartları, yazılım mimarisi prensipleri, sürüm kontrolü uygulamaları ve test stratejileri ayrıntılı biçimde ele alınmaktadır.

8.1. Değişiklik Yönetimi Süreci

8.1.1. Değişiklik Talep Sınıflandırması

Sistemin karmaşık ve çok bileşenli yapısı göz önünde bulundurularak, geliştirme sürecinde ortaya çıkan her değişiklik talebi dört kategoride sınıflandırılmıştır. Hata düzeltme (bug fix) kategorisi, sistemin belgelenmiş davranışından sapan ve kullanıcı deneyimini olumsuz etkileyen sorunları kapsamaktadır. İşlevsellik genişletme (feature addition) kategorisi, başlangıçta kapsam dışında tutulan ancak değerlendirme sürecinde gerekli görülen yeni yetenekleri içermektedir. Performans iyileştirme kategorisi, mevcut işlevselliği koruyarak yanıt süresi veya maliyet verimliliğini artıran değişiklikleri kapsamaktadır. Yapısal yeniden düzenleme (refactoring) kategorisi ise dış davranışı değiştirmeksizin kod okunabilirliğini ve sürdürülebilirliğini artıran değişiklikleri içermektedir.

Hata düzeltmeleri ve performans iyileştirmeleri öncelikli olarak ele alınmış; işlevsellik genişletmeleri ise yinelemeli geliştirme döngüleri içinde planlanarak gerçekleştirilmiştir. Bu önceliklendirme, çalışan sistemin kararlılığını korurken yeni özellik ekleme esnekliğini güvence altına almıştır.

8.1.2. Gerçekleştirilen Başlıca Değişiklikler

Geliştirme sürecinde dört kritik değişiklik kararı alınmış ve uygulanmıştır. Bu kararların her biri, ampirik gözlem ve test sonuçlarına dayalı olarak gerekçelendirilmiştir.

Tablo 8.1. Geliştirme Sürecindeki Temel Değişiklik Kararları

Bileşen	Başlangıç Değer	Son Değer	Gerekçe
Benzerlik Eşiği	$\tau = 0.30$	$\tau = 0.45$	Kapsam içi %30 yanlış ret gözlemlendi
Few-Shot k	k = 5 örnek	k = 3 örnek	Bağlam penceresi sınırı aşıyordu
Oturum Geçmişi	Sınırsız mesaj	Son 5 mesaj	Uzun sohbette token taşması
ChromaDB Yapısı	Tek koleksiyon	Belge başına koleksiyon	Silme işlemi çakışma yaratıyordu
Arayüz Çerçevesi	Gradio + Flask	Yalnızca Flask	Gradio script enjeksiyonunu engelledi

Tüm bu değişiklikler, değişiklik tarihi ve gerekçesiyle birlikte kod içi yorum satırlarında ve geliştirme günlüğünde belgelenmiştir. Bu belgeleme yaklaşımı, projenin ilerleyen aşamalarında veya başka geliştiriciler tarafından sürdürülmesi durumunda tasarım kararlarının izlenebilirliğini güvence altına almaktadır.

8.2. Sürüm Kontrolü ve Kod Yönetimi

8.2.1. Git Dal Stratejisi

Proje, Git tabanlı sürüm kontrolü ile yönetilmiştir. Geliştirme süreci boyunca üç katmanlı dal stratejisi izlenmiştir. main dalı yalnızca kararlı ve test edilmiş kodu barındırmaktadır; doğrudan bu dala commit yapılmamış, değişiklikler yalnızca gözden geçirme sonrasında birleştirilmiştir. develop dalı aktif geliştirme çalışmalarını kapsamakta; bu dal üzerinde özellik geliştirme tamamlandıkça main'e birleştirme yapılmaktadır. feature dalları ise deneysel bileşen geliştirmeleri için kullanılmış;

başarıyla tamamlanan özellikler develop'a, başarısız denemeler ise arşivlenerek silinmiştir.

Her commit mesajı değişikliğin türünü ve kapsamını açıkça belirtecek biçimde yazılmıştır. fix: SQL normalizasyonunda ILIKE hatası giderildi, feat: çoklu belge dinamik k hesaplama eklendi, refactor: RagManager belge başına koleksiyon mimarisine geçildi gibi örnek tabanlı bir format benimsenmiştir. Bu yaklaşım, commit geçmişinin okunabilirliğini ve filtrelenebilirliğini artırmaktadır.

8.2.2. Yedekleme ve Süreklilik

Sistem Google Colab ortamında Jupyter Notebook formatında geliştirilmiştir. Her geliştirme oturumu sonunda güncel sürüm Google Drive'a yedeklenmiş; kritik kilometre taşlarında sürüm numaralandırılmış arşivler oluşturulmuştur. Bu yaklaşım hem sürüm izlenebilirliğini hem de donanım arızası veya oturum sonlanması gibi beklenmedik durumlarda iş sürekliliğini güvence altına almaktadır.

8.3. Mühendislik Standartları ve Kod Kalitesi

8.3.1. Python Kod Yazım Standardı

Geliştirme sürecinde PEP 8 Python kod yazım standardı esas alınmıştır. Değişken ve fonksiyon adlandırmasında snake_case kuralı uygulanmış; anlamlı ve açıklayıcı isimler tercih edilerek tek harfli değişken kullanımından kaçınılmıştır. Sınıf adları PascalCase biçiminde yazılmıştır. Satır uzunluğu 100 karakter ile sınırlandırılmıştır. Tüm sınıflar ve genel işlevler için docstring biçiminde belgeleme yazılmıştır.

Modül düzeyinde erken dönüş (early return) prensibi iç içe geçmiş koşul bloklarının derinliğini azaltmak için uygulanmıştır. Sabitlerin modül başında büyük harfli değişkenler olarak tanımlanması; örneğin MAX_TOTAL_CHUNKS = 20, SIMILARITY_THRESHOLD = 0.45 şeklinde hem okunabilirliği artırmakta hem de değerlerin tek noktadan güncellenmesini kolaylaştırmaktadır.

8.3.2. Hata Yönetimi Politikası

Sistemin tüm harici etkileşim noktaları kapsamlı hata yönetimiyle korunmaktadır. OpenAI ve Google Gemini API çağrıları; ağ hatası, kimlik doğrulama hatası, hız sınırı aşımı ve geçersiz yanıt senaryolarını ayrı ayrı ele alan try-except blokları içinde sarmalanmıştır. Her hata durumunda kullanıcıya teknik hata mesajı yerine anlaşılır bir Türkçe geri bildirim iletilmektedir.

DuckDuckGo arama hatası durumunda sistem sessizce saf BDM yanıtına geçmekte ve kullanıcıya aramanın başarısız olduğunu bildirmektedir. SQL yürütme hataları öz-onarım mekanizmasına yönlendirilmekte; onarım da başarısız olursa "Bu sorgu şu anda çalıştırılmıyor" biçiminde bir açıklama döndürülmektedir. Belge yükleme hataları ise MIME türü doğrulama katmanında durdurulmakta ve kullanıcıya desteklenen formatlar listelenmektedir.

8.3.3. Güvenlik Standartları

API anahtarları hiçbir koşulda kaynak koda, Jupyter Notebook hücrelerine veya versiyon kontrol sistemine gömülmemektedir. Google Colab ortamında `userdata.get()`, yerel geliştirme ortamında `os.environ.get()` yöntemiyle okunmaktadır. Bu yaklaşım kimlik bilgilerinin herkese açık depolara sızmasını yapısal olarak engellemektedir.

Veritabanı sorguları SQLAlchemy'nin parametrelili sorgu yapısıyla oluşturulmakta; kullanıcı girdisi asla doğrudan SQL dizesine eklenmemektedir. Kullanıcı tarafından yüklenen dosyalar işlenmeden önce MIME türü doğrulamasından geçirilmekte; yalnızca `application/pdf`, `application/vnd.ms-excel` ve `application/vnd.openxmlformats-officedocument.spreadsheetml.sheet` türlerine izin verilmektedir. Dosya adları özel karakterlerden arındırılarak yükleme dizini dışına çıkış (path traversal) saldırıları önlenmektedir.

8.3.4. Bağımlılık Yönetimi

Tüm Python paket sürümleri requirements.txt dosyasında tam sürüm numarasıyla sabitlenmiştir. Bu yaklaşım, farklı ortamlarda tutarlı çalışma davranışını güvence altına almakta ve sürüm güncelleme kaynaklı kırılmaları önlemektedir. Bağımlılıklar minimum gereksinim ilkesiyle seçilmiş; yalnızca işlevsel bir gereksinimi karşılayan paketler eklenerek gereksiz bağımlılık ağırlığından kaçınılmıştır.

8.4. Yazılım Mimarisi Standartları

8.4.1. SOLID Prensipleri

Sistem tasarımında nesne yönelimli programlamanın temel prensipleri olan SOLID kuralları esas alınmıştır.

Tek Sorumluluk Prensipleri (SRP) çerçevesinde her sınıf ve işlev yalnızca tek bir görevi yerine getirmektedir: RagManager belge işleme ve vektör geri almayı, intent_classify() niyet tespitini, text_to_sql() SQL üretimini, llm_invoke_tracked() token sayımını üstlenmektedir. Bu ayrışım, her bileşenin bağımsız olarak test edilmesini ve değiştirilmesini kolaylaştırmaktadır.

Açık/Kapalı Prensipleri (OCP) doğrultusunda yeni bir BDM sağlayıcısı eklemek mevcut kodu değiştirmeksizin llm_olustur() fabrika işlevinin genişletilmesiyle mümkündür. Yeni bir belge formatı desteği eklemek ise yalnızca RagManager içinde yeni bir yükleyici dalı gerektirmekte; varolan mantığa dokunmamaktadır.

Bağımlılık Tersine Çevirme Prensipleri (DIP) gereği üst düzey modüller somut uygulamalara değil, LangChain soyutlamalarına bağımlıdır. ChatOpenAI veya ChatGoogleGenerativeAI nesneleri aynı BaseChatModel arayüzüyle kullanılmakta; bu sayede model değişikliği üst katmanda hiçbir değişiklik gerektirmemektedir.

8.4.2. Uygulanan Tasarım Desenleri

Sistem genelinde dört tasarım deseni bilinçli olarak uygulanmıştır.

Fabrika Deseni, `llm_olustur(model_adi)` işleviyle BDM örneği oluşturmada kullanılmaktadır. İstemci kodu hangi sağlayıcının kullanıldığından habersizdir; yalnızca fabrika işlevine model adını iletmekte ve standart bir BDM nesnesi almaktadır.

Strateji Deseni, niyet tabanlı işlem hattı seçiminde uygulanmaktadır. Her niyet kategorisi (DB_QUERY, DOCUMENT, SEARCH, GENERAL, META) bağımsız bir strateji işlevi olarak tanımlanmış; merkezi yönlendirici bu işlevlerden birini seçerek çağırılmaktadır.

Gözlemci Deseni, token sayacı sarmalayıcısında kullanılmaktadır. `llm_invoke_tracked()` her BDM çağrısını izleyerek token ve maliyet bilgisini hem global sayaca hem de aktif oturumun sayacına iletmektedir.

Kayıt Defteri Deseni (Registry), RagManager'ın belge deposunda uygulanmaktadır. Her yüklü belge benzersiz adıyla kayıt defterine eklenmekte; sorgulama ve silme işlemleri bu merkezi kayıt üzerinden yönetilmektedir.

8.5. Test ve Doğrulama Süreçleri

8.5.1. Test Katmanları

Sistem iki farklı test katmanında doğrulanmıştır. Bileşen testlerinde her işlem hattı bağımsız olarak değerlendirilmiştir: Metin-SQL modülü yalnızca veritabanı bağlantısıyla, RAG yöneticisi yalnızca ChromaDB ile, niyet sınıflandırıcı yalnızca BDM API'siyle ayrı ayrı test edilmiştir. Bu izolasyon, bir bileşendeki hatanın diğer bileşenlere karışmasını önlemektedir. Uçtan uca testlerde gerçek kullanıcı senaryoları simüle edilerek tam iş akışı değerlendirilmiştir: kullanıcı girdisi Flask API'ye

iletilmiş, niyet sınıflandırma ve ilgili işlem hattının tetiklenmesi, yanıtın ve token bilgisinin döndürülmesi kontrol edilmiştir.

8.5.2. Test Veri Kümeleri

Test veri kümeleri el ile oluşturulmuş; her test sorusu için beklenen çıktı önceden belirlenmiştir. Bu yaklaşım, otomatik regresyon testi altyapısı kurulmaksızın deterministik doğrulama yapılmasına olanak tanımaktadır. Metin-SQL için 30, RAG için 20 ve niyet sınıflandırma için 50 test örneği hazırlanmıştır. Her test oturumu $temperature=0$ parametresiyle çalıştırılarak sonuçların tekrarlanabilirliği güvence altına alınmıştır.

8.5.3. Regresyon Kontrolü

Bir bileşende değişiklik yapıldığında ilgili test kümesi yeniden çalıştırılmış ve başarımların değerlerinin gerilememesi kontrol edilmiştir. $\tau=0.45$ eşiği değişikliğinden sonra tüm 20 RAG test sorusu yeniden test edilmiş; kapsam dışı ret oranının %100 düzeyinde korunduğu doğrulanmıştır. $k=3$ few-shot değişikliğinden sonra 30 SQL test sorusu yeniden çalıştırılmış; %90 genel doğruluk değerinin bozulmadığı teyit edilmiştir.

BÖLÜM 9. TİCARİLEŞME PLANI

Akademik bir prototip olarak geliştirilen bu sistemin gerçek kurumsal ortamlara taşınabilmesi için kapsamlı bir ticarileşme stratejisi oluşturulmuştur. Bu bölümde pazar analizi, ürün olgunlaştırma yol haritası, iş modeli, finansman stratejisi ve risk yönetimi ele alınmaktadır.

9.1. Pazar Analizi ve Hedef Kitle

9.1.1. Pazar Büyüklüğü ve Fırsat

Türkiye'de 208 üniversite ve bu üniversitelerde kayıtlı yaklaşık 8,5 milyon öğrenci bulunmaktadır. Buna ek olarak 200.000'den fazla akademisyen ve on binlerce idari personel bilgi yönetim sistemlerine rutin olarak ihtiyaç duymaktadır. YÖK istatistiklerine göre her yıl ortalama 700.000 yeni öğrenci yükseköğretime kaydolmaktadır.

Küresel ölçekte yükseköğretim yapay zekâ pazarının 2025 yılında 3,68 milyar dolar olduğu tahmin edilmekte; 2030 yılına kadar %45 yıllık büyüme oranıyla 20 milyar doları aşması beklenmektedir. Türkiye, bu global büyümede Orta Doğu, Avrupa ve Orta Asya'ya açılan bir köprü konumunda yer almaktadır.

9.1.2. Hedef Pazar Kademeleri

Hedef pazar üç kademeli bir yapıda planlanmıştır. Birincil hedef pazar, Türk devlet ve vakıf üniversiteleridir; özellikle mevcut OBS sistemlerini doğal dil arayüzüyle zenginleştirmek isteyen orta ve büyük ölçekli kurumlar öncelikli hedef grubu oluşturmaktadır. Pilot aşamada iki veya üç üniversiteyle sözleşme imzalanması hedeflenmektedir.

İkincil hedef pazar, özel yükseköğretim kurumları ve mesleki eğitim merkezleridir. Bu kurumlar rekabetçi baskıyla dijital dönüşüme daha hızlı uyum sağlama eğilimindedir; satış döngüsü kamu üniversitelerine kıyasla daha kısadır. Üçüncül hedef pazar ise üniversite bilgi yönetim altyapısına benzer gereksinimleri olan kamu kurumları, belediyeler ve büyük şirketlerin insan kaynakları ve bilgi yönetim birimleridir.

9.1.3. Rekabet Analizi

Mevcut çözümler iki ana kategoride değerlendirilebilir. Genel amaçlı chatbot platformları olan ChatGPT, Microsoft Copilot ve Google Gemini, güçlü dil anlama yeteneklerine sahip olmakla birlikte kurumsal veritabanlarına bağlanamamakta ve halüsinasyon riskine karşı savunmasızdır. Mevcut üniversite bilgi sistemleri olan ÜNIFANS ve çeşitli OBS platformları ise kapsamlı veri yönetimi sunarken doğal dil arayüzünden yoksundur; kullanıcıların formlara ve menülere bağımlı kalmasına neden olmaktadır. Önerilen sistem bu iki kategorinin güçlü yanlarını bir araya getirerek mevcut çözüm boşluğunu doldurmaktadır.

9.2. Ürün Olgunlaştırma Yol Haritası

Prototipten ticarileşmeye geçiş üç aşamada planlanmıştır.

Tablo 9.1. Ürün Olgunlaştırma Aşamaları

Aşama	Süre	Temel Çıktılar	Hedef
MVP	0-6 ay	Gerçek OBS bağlantısı, BERTurk gömme, kimlik doğrulama, React+FastAPI arayüzü	1 pilot üniversite, 100-500 kullanıcı
Pilot	6-18 ay	Eşik optimizasyonu, analitik paneli, çok belgeli akıl yürütme, 5 üniversitede paralel test	5 üniversite, 2.500+ kullanıcı
Ticari	18-36 ay	Çok kiracılı bulut mimarisi, mobil uygulama, KVKK/GDPR uyumluluk belgesi, standart API katmanı	20+ üniversite, SaaS gelir

9.3. İş Modeli ve Gelir Stratejisi

9.3.1. Gelir Akışları

Ticari sürdürülebilirlik için dört gelir akışı tasarlanmıştır. Birinci ve temel akış, Hizmet Olarak Yazılım (SaaS) abonelik modelidir. Kurumsal abonelik ücreti kullanıcı sayısına ve aylık sorgu hacmine göre kademelendirilen bir yapıda sunulacaktır: 500 kullanıcıya kadar aylık 2.000-5.000 TL, 500-5.000 kullanıcı için aylık 10.000-25.000 TL ve 5.000'den fazla kullanıcı için özel fiyatlandırma uygulanacaktır.

İkinci akış kurulum ve entegrasyon hizmetleridir. Mevcut OBS sistemleriyle entegrasyon, veri taşıma ve personel eğitimi kapsamında tek seferlik kurulum bedeli alınacaktır. Üçüncü akış yıllık bakım ve teknik destek sözleşmeleridir; bu hizmet yıllık lisans ücretinin %20'si oranında fiyatlandırılacaktır. Dördüncü akış ise kuruma özgü özelleştirme ve yapay zekâ danışmanlığı kapsamında saat bazlı veya proje bazlı ücretlendirilecektir.

9.3.2. Beş Yıllık Gelir Projeksiyonu

Tablo 9.2. Beş Yıllık Gelir Projeksiyonu (Yalnızca SaaS Geliri)

Metrik	Yıl 1	Yıl 2	Yıl 3	Yıl 4	Yıl 5
Üniversite Sayısı	2	5	12	25	45
Aktif Kullanıcı	500	2.000	6.000	15.000	30.000
Tahmini Ciro (TL)	120.000	480.000	1.440.000	3.600.000	7.200.000

9.4. Finansman ve Büyüme Stratejisi

9.4.1. Başlangıç Finansmanı

Projenin MVP aşaması için gereken başlangıç finansmanı iki temel kaynaktan sağlanması planlanmaktadır. TÜBİTAK 1512 Teknogirişim Sermaye Desteği Programı kapsamında 1.500.000 TL'ye kadar hibe desteği hedeflenmektedir. Bu program, üniversite kökenli teknoloji girişimlerini öncelikli olarak desteklemekte

olup akademik danışmanlık ilişkisi ve patent başvurusu güçlü bir avantaj oluşturmaktadır.

KOSGEB Girişimcilik Destek Programı aracılığıyla ek donanım ve personel giderleri karşılanacaktır. Sakarya Üniversitesi Teknoloji Transfer Ofisi (TTO) üzerinden teknoloji lisanslama geliri ve spin-off şirketi kuruluşu için kurumsal destek alınması planlanmaktadır.

9.4.2. Büyüme Aşamaları

Pilot aşamada elde edilen kullanıcı verileri ve gelir referansları melek yatırımcı turuna zemin hazırlayacaktır. 5-7 üniversitede aktif kullanım kanıtlandıktan sonra 3-5 milyon TL değerlemesiyle melek turu gerçekleştirilmesi hedeflenmektedir. Bu finansmanla geliştirme ekibi büyütülecek ve satış altyapısı kurulacaktır.

Uzun vadede Türkiye pazarında yeterli ölçeğe ulaşıldıktan sonra KKTC ve Balkan ülkelerindeki Türkçe konuşan akademik kurumlara uluslararası açılım planlanmaktadır. Çok dilli gömme modeli altyapısı, Arapça ve Farsça dillerine genişlemeyi teknik açıdan mümkün kılmaktadır.

9.5. Fikri Mülkiyet Stratejisi

Sistemin üç özgün bileşeni fikri mülkiyet koruması kapsamında değerlendirilmektedir: τ eşik kapısıyla uygulanan sıfır-halüsinasyon temelleme politikasının somut gerçekleştirimi; dinamik k formülüne dayalı çoklu belge RAG mimarisi ve tek adımlı öz-onarım döngüsüne sahip few-shot Metin-SQL hattının tasarımı.

Kısa vadede bu bileşenler için Türk Patent ve Marka Kurumu (TÜRKPATENT) nezdinde faydalı model başvurusu planlanmaktadır. Faydalı model tescili, patent başvurusuna kıyasla daha kısa süre ve daha düşük maliyetle gerçekleştirilebilmektedir. Yazılım telif hakkı tescili bu süreçle paralel yürütülecektir.

Akademik yayınlar aracılığıyla uluslararası öncelik tarihi tesis edilmesi ek bir koruma katmanı sağlayacaktır.

9.6. Risk Analizi ve Azaltma Stratejileri

Ticarileşme sürecindeki başlıca riskler dört kategoride değerlendirilmiştir.

Tablo 9.3. Risk Matrisi ve Azaltma Stratejileri

Risk Kategorisi	Risk Tanımı	Etki	Azaltma Stratejisi
Teknik	BDM sağlayıcıları API fiyat/politika değişikliği	Yüksek	Çok sağlayıcı mimarisi; Llama/Mistral açık kaynak yedek
Pazar	OBS sistemleriyle entegrasyon güçlüğü	Orta	Standart REST API katmanı; hibrit bulut/yerinde seçenek
Düzenleyici	KVKK/GDPR kişisel veri gereklilikleri	Yüksek	Yerinde (on-premise) dağıtım seçeneği; veri yerelleştirme
Rekabet	Büyük teknoloji şirketlerinin benzer çözüm geliştirmesi	Orta	Uzun vadeli kurumsal sözleşmeler; alan özgü derinleşme
Finansal	Kamu üniversitelerinde uzun bütçe onay süreçleri	Orta	Pilot için ücretsiz deneme; başarı sonrası sözleşme modeli

KAYNAKLAR

- [1] T. Yu, R. Zhang ve K. Yang, «Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Text-to-SQL Tasks,» %1 içinde *Proc. EMNLP*, 2018.
- [2] H. Weld, X. Huang, S. Long, J. Poon ve S. C. Han, «A Survey of Joint Intent Detection and Slot Filling Models in Natural Language Understanding,» *ACM Computing Surveys*, pp. 1-38, 2022.
- [3] R. Sun, S. Arik ve R. Sinha, «SQLPrompt: In-Context Text-to-SQL with Minimal Labeled Data,» 2023.
- [4] K. Shuster, S. Poff, M. Chen, D. Kiela ve J. Weston, «Retrieval Augmentation Reduces Hallucination in Conversation,» %1 içinde *Findings of EMNLP*, 2021.
- [5] T. Scholak, N. Schucher ve D. Bahdanau, «PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models,» %1 içinde *Proc. EMNLP*.
- [6] I. C. Rafidhul Haque, «Implementation of Retrieval-Augmented Generation (RAG) and Large Language Models for a Document and Tabular-Based Chatbot System,» *Journal of Electronics Technology Exploration*, pp. 19-23, 2025.
- [7] M. Pourreza ve D. Rafiei, «DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction,» 2023.
- [8] L. Pasquarelli, C. Koutchme ve A. Hellas, «Comparing the Utility, Preference, and Performance of Course Material Search Functionality and Retrieval-Augmented Generation Large Language Model AI Chatbots in Information-Seeking Tasks,» 2024.
- [9] OpenAI, «GPT-4 Technical Report,» 2024.
- [10] L. Nan, Y. Zhao ve W. Zou, «Enhancing Few-Shot Text-to-SQL Capabilities of Large Language Models: A Study on Prompt Design Strategies,» 2023.
- [11] T. Ma, «REBot: From RAG to CatRAG with Semantic Enrichment and Graph Routing,» 2025.

- [12] J. Liu, D. Shen, Y. Zhang, B. Dolan, L. Carin ve W. Chen, «What Makes Good In-Context Examples for GPT-3?,» %1 içinde *Proc. DeeLIO Workshop at ACL*, 2022.
- [13] P. Lewis, E. Perez ve A. Piktus, «Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,» %1 içinde *Advances in Neural Information Processing Systems*, 2020.
- [14] Z. Ji, N. Lee ve R. Frieske, «Survey of Hallucination in Natural Language Generation,» *ACM Computing Surveys*, pp. 1-38, 2023.
- [15] Y. Gao, Y. Xiong ve X. Gao, «Retrieval-Augmented Generation for Large Language Models: A Survey,» 2023.
- [16] H. Ding, L. Pang ve Z. Wei, «Rowen: Adaptive Retrieval-Augmented Generation for Hallucination Mitigation in LLMs,» %1 içinde *SIGIR-AP 2025: Proceedings of the 2025 Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region*, 2025.
- [17] J. Devlin, M.-W. Chang, K. Lee ve K. Toutanova, «BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding,» %1 içinde *Proc. NAACL-HLT*, 2019.
- [18] R. Dayarathne, U. Ranaweera ve U. Ganegoda, «Comparing the Performance of LLMs in RAG-Based Question-Answering: A Case Study in Computer Science Literature,» %1 içinde *Proc. AIET, Springer*, 2025.
- [19] C. Burgan, J. Kowalski ve W. Liao, «Developing a Retrieval Augmented Generation (RAG) Chatbot App Using Adaptive Large Language Models and LangChain Framework,» %1 içinde *Proc. West Virginia Academy of Science*, 2024.
- [20] L. Bergeron, I. Buhnila ve J. François, «HalluGuard: Evidence-Grounded Small Reasoning Models to Mitigate Hallucinations in Retrieval-Augmented Generation,» 2025.
- [21] A. Asai, Z. Wu, Y. Wang, A. Sil ve H. Hajishirzi, «Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection,» %1 içinde *Proc. ICLR*, 2024.

- [22] G. C. López ve G. d. Chalendar, «Intent Recognition and Out-of-Scope Detection Using LLMs in Multi-Party Conversations,» %1 içinde *Proc. SIGDIAL*, 2025.
- [23] G. T. Google, R. Anil ve S. Borgeaud, «Gemini: A Family of Highly Capable Multimodal Models,» 2023.

ÖZGEÇMİŞ

Egemen Bozca, 2003 yılında doğmuştur. Sakarya Üniversitesi Bilgisayar Mühendisliği bölümünde lisans eğitimini sürdürmektedir.

Lise yıllarından itibaren yapay zekâ ve donanım alanlarında çeşitli ulusal ve uluslararası yarışmalara katılmıştır. RoboChallenge AirRace yarışmasında dünya birinciliği elde etmiş; Teknofest Liselerarası İHA Yarışması'nda Türkiye ikincisi, TÜBİTAK bünyesinde gerçekleştirilen projelerde ise Türkiye ikincisi ve üçüncüsü olmuştur. 2024 yılında Teknofest Üniversite Öğrencileri Araştırma Proje Yarışması finaline katılmıştır.

Mobil uygulama geliştirme alanında React Native teknolojisi ile geliştirdiği dört uygulama Google Play'de yayınlanmaktadır. 2023 yılında Smart Software A.Ş. bünyesinde yazılım ve donanım geliştirme alanlarında staj yapmıştır.

Bitirme çalışması kapsamında ajansal RAG mimarisi, büyük dil modelleri ve Metin-SQL teknolojilerini bütünleştiren akademik bir chatbot sistemi geliştirmiştir. Python, LangChain, ChromaDB ve Flask teknolojilerini kullanan bu çalışmada halüsinasyon önleme, niyet sınıflandırma ve gerçek zamanlı maliyet takibi konularında özgün katkılar ortaya koymuştur.

BSM 498 BİTİRME ÇALIŞMASI DEĞERLENDİRME VE SÖZLÜ SINAV TUTANAĞI

KONU: EĞİTİM ALANINDA ÖZELLEŞTİRİLMİŞ RAG TABANLI AKADEMİK
CHATBOT

ÖĞRENCİLER (Öğrenci No/AD/SOYAD): B221210037/EGEMEN/BOZCA

Değerlendirme Konusu	İstenenler	Not Aralığı	Not
Yazılı Çalışma			
Çalışma klavuza uygun olarak hazırlanmış mı?	x	0-5	
Teknik Yönden			
Problemin tanımı yapılmış mı?	x	0-5	
Geliştirilecek yazılımın/donanımın mimarisini içeren blok şeması (yazılımlar için veri akış şeması (dfd) da olabilir) çizilerek açıklanmış mı?			
Blok şemadaki birimler arasındaki bilgi akışına ait model/gösterim var mı?			
Yazılımın gereksinim listesi oluşturulmuş mu?			
Kullanılan/kullanılması düşünülen araçlar/teknolojiler anlatılmış mı?			
Donanımların programlanması/konfigürasyonu için yazılım gereksinimleri belirtilmiş mi?			
UML ile modelleme yapılmış mı?			
Veritabanları kullanılmış ise kavramsal model çıkarılmış mı? (Varlık ilişki modeli, noSQL kavramsal modelleri v.b.)			
Projeye yönelik iş-zaman çizelgesi çıkarılarak maliyet analizi yapılmış mı?			
Donanım bileşenlerinin maliyet analizi (prototip-adetli seri üretim vb.) çıkarılmış mı?			
Donanım için gerekli enerji analizi (minimum-uyku-aktif-maksimum) yapılmış mı?			
Grup çalışmalarında grup üyelerinin görev tanımları verilmiş mi (iş-zaman çizelgesinde belirtilebilir)?			
Sürüm denetim sistemi (Version Control System; Git, Subversion v.s.) kullanılmış mı?			
Sistemin genel testi için uygulanan metotlar ve iyileştirme süreçlerinin dökümü verilmiş mi?			
Yazılımın sızma testi yapılmış mı?			
Performans testi yapılmış mı?			
Tasarımın uygulamasında ortaya çıkan uyumsuzluklar ve aksaklıklar belirtilerek çözüm yöntemleri tartışılmış mı?			
Yapılan işlerin zorluk derecesi?	x	0-25	
Sözlü Sınav			
Yapılan sunum başarılı mı?	x	0-5	
Soruları yanıtlama yetkinliği?	x	0-20	
Devam Durumu			
Öğrenci dönem içerisindeki raporlarını düzenli olarak hazırladı mı?	x	0-5	
Diğer Maddeler			
Toplam			

DANIŞMAN (JÜRİ ADINA):

DANIŞMAN İMZASI: